OPERAÇÕES NO CONEX

chuveiros.tex

OPERAÇÕES NO CONEX

Simulações híbridas de
chuveiros atmosféricos extensos

```
$ O que há de se explorar nesse caderno?
> install CONEX
> simule chuveiros
> ganhe o nobel.
```

01 / MAIS SOBRE O CADERNO

Esse arquivo é um guia que vai desde a instalação do CONEX até as suas primeiras simulações. Vou explorar as configurações básicas do software e o significado de algumas flags mais profundas. Espero que se torne um arquivo útil de primeira viagem em simulações de chuveiros de partículas.

"Enquanto eu tiver perguntas e não houver resposta continuarei a escrever."

00 / COMECE POR AQUI Guia / APOEMA

Mapa do documento

Temas e páginas

OBJETIVO Localizar os assuntos e acessar a primeira página de cada tema.

1 Percurso: Como usar este guia? 3

2 Fundamentos: O que está sendo simulado? 5

3 Ambiente: Antes de instalar 7

4 Instalação: Instalar o CONEX 14

5 Primeiro teste: Um chuveiro verificável 17

6 Configuração: Energia e amostragem 23

7 Análise em Python: Ler os resultados com PyROOT 29

8 Análise em C++: Ler os resultados com C++ 38

9 Rotina: Uso cotidiano 47

10 Consulta: Glossário clicável 50

11 Fontes: Referências e aprofundamento 53

12 Consulta: Comandos Linux 55

Este sumário compacto inclui os temas. As subseções internas permanecem numeradas nas páginas de conteúdo.

01 / PERCURSO
Guia / APOEMA

Como usar este guia?

Uma operação precisa ter significado

OBJETIVO Escolher um caminho de leitura e reconhecer o nível de verificação de cada procedimento.

Este caderno conduz da preparação do ambiente à simulação e comparação de chuveiros atmosféricos extensos. Leia os fundamentos antes de executar; depois siga o primeiro teste e escolha uma das duas rotas de análise. Os blocos de terminal usam Bash em Linux, inclusive dentro do WSL 2. Comandos para PowerShell são identificados explicitamente. Não copie o símbolo de prompt nem quebras visuais de linha do PDF.

1.1 De onde veio o conteúdo

Esse guia surge como contribuição dos trabalhos de alunos do grupo APOEMA após anos de operações no CONEX. Agradeço principalmente ao Marcos Olegário que contribuiu com o percurso de instalação e a explicação inicial das árvores de saída.

Este material surge de primeiros princípios quando eu ainda estava aprendendo a realizar simulações com o CONEX. A falta de documentação expressa, como é para o CORSIKA, torna o CONEX mais difícil de se trabalhar, muito embora seja mais simplificado.

Portanto, compilei neste material as principais práticas e configurações para qualquer um que inicie suas simulações com chuveiros no CONEX consiga também seguir, sem precisar passar por anos de estresse e buscas minuciosas. Espero que seja de grande valia.

1.2 Identificação da distribuição

> ATENÇÃO /
>
> Estarei tratando neste documento da versão 7.8 do CONEX. No momento em que estiver lendo, pode estar desatualizado.

1.3 Rotas de leitura

Iniciante: leia os temas 2 a 6 nessa ordem e então escolha **uma** rota: Python com PyROOT, seção 7, ou macros C++, seção 8. As duas ensinam a abrir arquivos, ler árvores e branches, representar perfis, fazer histogramas, comparar conjuntos e consultar interações líderes. Não é necessário ler ou executar ambas.

Quem já compila pode ir ao tema 5 e consultar o tema 6 durante o uso. Depois da rota escolhida, siga para Uso cotidiano, seção 9. O glossário e a consulta de comandos ficam ao final, com links que funcionam sem internet.

REFERÊNCIA / README, Makefile, src/conex_sub.F:1053 e src/CxRoot.cc:335; tags públicas do cxroot [5].

01 / PERCURSO [continuação]
Guia / APOEMA

Como usar este guia?

Baixar os arquivos que acompanham o guia

OBJETIVO Obter os códigos e conservar a organização esperada pelos comandos.

Os exemplos estão no repositório davisilva169/CONEX-guide, no GitHub. Abra esse link e escolha **Code** > **Download ZIP**. Extraia o pacote em uma pasta de trabalho no Linux ou no WSL, de preferência com caminho sem espaços. Não é necessário ter uma conta no GitHub para baixar.

1.4 O que baixar

- `python`: o script Python com as operações de análise.
- `macros`: as cinco alternativas C++/ROOT. Baixe a pasta inteira: `histogram.C` usa `compare.C`.
- `scripts`: geração da pequena campanha e reprodução dos exemplos.
- `results`: instruções para gerar os arquivos ROOT, gráficos de referência, resumo e procedência.

O guia está disponível ali em PDFs em português e em inglês. Os caminhos e comandos dos exemplos usam os mesmos nomes em inglês nas duas edições. Os fontes LaTeX permanecem no projeto editável entregue à parte. CONEX, tabelas e dependências são obtidos e instalados pelo percurso das próximas seções.

1.5 De onde executar os comandos

Quando pedirmos a *raiz do repositório*, entre na pasta extraída que contém `python`, `macros` e `scripts`. Ajuste o caminho abaixo ao local escolhido:

entrar na pasta dos exemplos — bash / cód. 1.1

```bash
cd "$HOME/caminho/para/CONEX-guide-main"
ls python macros scripts results
```

Preserve essa estrutura. Se baixar um arquivo isolado pelo link do guia, abra sua página no GitHub e use **Download raw file**; salve-o na pasta indicada. Para começar, baixar o ZIP completo evita faltar uma dependência.

> **ATENÇÃO / Códigos para baixar; dados para gerar**
>
> Os links junto dos blocos levam aos códigos necessários. Os nomes de arquivos `.root` levam às respectivas receitas em `results`: os eventos não são disponibilizados para download. Primeiro produza a saída em sua instalação; depois ajuste as variáveis `ARQ`, `A`, `B` ou `LIDER` ao caminho real. Os gráficos e o resumo de referência podem ser baixados para comparação.

02 / FUNDAMENTOS Guia / APOEMA

O que está sendo simulado?

A cascata e a aproximação híbrida

OBJETIVO Relacionar o método computacional às grandezas que aparecem na saída.

O CONEX é um software de simulação híbrida que combina o método de Monte Carlo e equações de cascata para simular chuveiros atmosféricos extensos.

Um raio cósmico primário pode interagir com núcleos do ar e produzir partículas secundárias. Novas interações e decaimentos desenvolvem um chuveiro atmosférico extenso. O CONEX calcula perfis ao longo do eixo do chuveiro, incluindo flutuações de um evento para outro.

Na parte de maior energia, o Monte Carlo sorteia processos individuais. Partículas abaixo dos limiares alimentam termos-fonte para equações de cascata: uma descrição numérica das populações em energia e profundidade. A combinação permite estudar perfis e depósito de energia com menor custo que acompanhar explicitamente todas as partículas. O artigo de Bergmann et al. apresenta o algoritmo e sua comparação com simulações completas [1]; o artigo inicial de Pierog et al. documenta a motivação e os primeiros resultados [2].

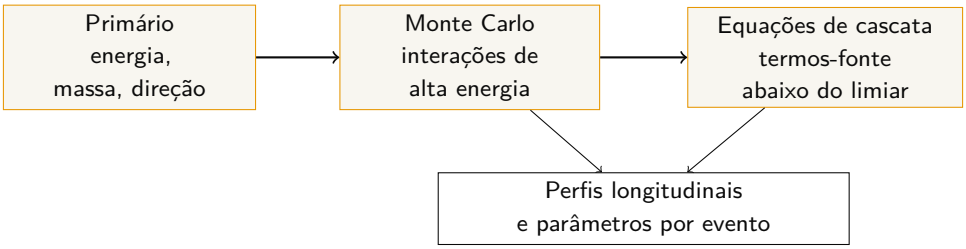


Figura 2.1: Representação esquemática dos processos envolvidos durante as simulações no CONEX.

> **CONCEITO 2.1 / Modelo de interação**
>
> Prescrição física para probabilidades e produção de secundárias nas colisões. Escolher um modelo modifica a física da simulação; não é apenas selecionar uma opção de velocidade.

2.1 O alcance da descrição 1D

A coordenada principal é a profundidade ao longo do eixo. O modo padrão do CONEX estudado não fornece uma distribuição lateral completa de partículas para um detector de superfície. Perfis e flutuações ajudam estudos de composição e técnicas de fluorescência, mas não substituem a propagação óptica, resposta instrumental ou reconstrução do experimento. Essa é uma limitação real do simulador e que deve estar evidente desde o início.

Há código para integrações com CORSIKA e opções tridimensionais, mas o executável independente `bin/conex2r` e o CMake de integração com CORSIKA 8 têm finalidades distintas. Não habilite macros de compilação para obter 3D como se fosse uma flag de execução validada neste guia.

REFERÊNCIA / README: INTRODUCTION; src/conex_sub.F (opções de pré-processador); CMakeLists.txt; [1, 2].

02 / FUNDAMENTOS [continuação] Guia / APOEMA

O que está sendo simulado?

Grandezas e unidades antes das flags

OBJETIVO Interpretar um perfil sem confundir energia, contagem, altitude e profundidade.

CONCEITO 2.2 / Primário

Partícula que inicia o chuveiro. O tipo, a energia total e a direção definem condições iniciais; diferentes eventos com essas condições ainda podem flutuar.

Grandeza	Significado e unidade
E	Energia total do primário; as flags recebem $\lg E = \log_{10}(E/\mathrm{eV})$, não E em GeV.
θ	Ângulo zenital em graus; zero corresponde ao caso vertical descendente usado nos exemplos.
X	Profundidade atmosférica ao longo do eixo (inclinada), em $\mathrm{g\,cm^{-2}}$.
H	Altitude em metros, distinta de X (vertical).
$N(X)$	Número de partículas carregadas acima dos cortes especificados. Não inclui fótons como cargas.
dE/dX	Depósito de energia por profundidade: $\mathrm{GeV/(g\,cm^{-2})}$.
$X_{\max}$	Profundidade do máximo do chuveiro (maior número de partículas secundárias produzidas) (ajuste Gaisser-Hillas, $\mathrm{g\,cm^{-2}}$.).

CONCEITO 2.3 / Profundidade atmosférica

É a massa de ar por área atravessada ao longo de uma trajetória: $X(s) = \int \rho(s')\,ds'$. Para uma atmosfera dada, H e X se relacionam, mas não são a mesma coordenada.

Altitude descreve posição geométrica; profundidade descreve material atravessado. Na aproximação plana, $X_{\mathrm{incl}} \simeq X_{\mathrm{vertical}}/\cos\theta$ para ângulos moderados. Perto do horizonte, curvatura e geometria importam: use as coordenadas calculadas pelo programa, não essa aproximação para substituir a atmosfera.

EXEMPLO 2.1 / Ler a escala de energia

`-e 17 -E 17` fixa $E = 10^{17}$ eV $= 10^{8}$ GeV. Já 19,5 corresponde a aproximadamente $3{,}16 \times 10^{19}$ eV. O ponto decimal em comandos é `19.5`; a vírgula é usada na prosa em português. O programa converte a energia em eV para GeV internamente em `GetEnergy`.

EXERCÍCIO 2.1 / Para praticar, sem gabarito

1. Converta 10^{18} eV em GeV e explique a diferença entre escrever 18 e 10^{18} na flag de energia.
2. Dois chuveiros têm o mesmo H e ângulos diferentes. Explique por que isso não fixa o mesmo X.
3. Por que um único perfil não representa as flutuações de um conjunto de chuveiros?

REFERÊNCIA / README: descrição dos branches; src/CxRoot.cc:GetEnergy; definição física de profundidade e descrição do método em [1].

03 / AMBIENTE
Guia / APOEMA

Antes de instalar

Terminal, caminhos e ferramentas

OBJETIVO Saber onde um comando será executado e de onde vêm suas ferramentas.

Ao longo do guia vamos usar comandos de terminal para navegar entre pastas, conferir arquivos e executar programas. A tabela de comandos Linux ao final do guia reúne essas instruções e exemplos de consulta, como `ls`, `cd`, `cd ..` e `mkdir`. Consulte-a sempre que precisar lembrar a função de um comando.

O código do CONEX e os seus arquivos complementares podem ser obtidos diretamente na página do IAP.

Como o acesso a esse servidor é protegido por senha, o primeiro passo é abrir o seu navegador e acessar a URL: `https://web.iap.kit.edu/corsika/download/`.

Quando o navegador solicitar a autenticação, basta preencher com o usuário `corsika` e a senha que você recebe por e-mail ao solicitar da colaboração.

Por fim, para encontrar os arquivos do CONEX, basta clicar na pasta `conex/` listada no diretório principal e selecionar os pacotes desejados para download.

CONCEITO 3.1 / Terminal e shell

O terminal é a interface de texto. A shell interpreta os comandos digitados. Neste guia usamos Bash no Linux; PowerShell no Windows é outra shell, com sintaxe diferente.

CONCEITO 3.2 / Caminhos

Um caminho absoluto começa na raiz, como `/home/ana/conex`. Um caminho relativo, como `cfg/conex_sibyll.param`, depende de onde você está. É a sequência de caracteres que dá a localização exata de um arquivo ou uma pasta dentro de um sistema de arquivos. glossário

ATENÇÃO /

Nomes diferenciam maiúsculas e minúsculas no Linux.

Antes de instalar

Reconhecer o ambiente

OBJETIVO Conferir o sistema e as ferramentas antes de escolher a instalação.

Abra um terminal Linux. Vamos fazer um diagnóstico do ambiente antes da instalação, sem instalar nada. Realize a sequência de comandos do bloco abaixo e, se alguma ferramenta ainda não estiver instalada ou ativada, o comando correspondente pode falhar.

reconhecimento bash / cód. 3.1

```bash
pwd # mostra a pasta atual.
uname -a # informa o sistema, o kernel e a arquitetura.
cat /etc/os-release # identifica a distribuição Linux e sua versão.
command -v gcc g++ gfortran make root root-config # verifica se as ferramentas são encontradas e mostra seus caminhos.
gcc --version # mostra a versão do compilador C.
gfortran --version # mostra a versão do compilador Fortran.
root-config --version # mostra a versão do ROOT.
root-config --cflags --libs # mostra as opções necessárias para compilar e ligar programas às bibliotecas do ROOT.
```

Esse passo é importante para identificar o sistema utilizado e conferir quais ferramentas já estão disponíveis, suas versões e seus caminhos. Essas informações ajudam a escolher dependências compatíveis e reconhecer o que ainda precisa ser instalado. Se algum comando não for encontrado, a ferramenta pode estar ausente ou apenas precisar que seu ambiente seja ativado.

> CONCEITO 3.3 / Compilador
>
> Traduz código-fonte para código executável. O pacote combina C, C++ e Fortran; a disponibilidade de um compilador C++ sozinho não resolve as três linguagens.

> CONCEITO 3.4 / Biblioteca
>
> Conjunto de funções usado por outros programas. As bibliotecas dos modelos são carregadas na execução; compilar o executável não prova que um modelo possa ser carregado e inicializado.

03 / AMBIENTE [continuação] Guia / APOEMA

Antes de instalar

Onde cada camada entra

OBJETIVO Reconhecer o projeto, os arquivos gerados e as dependências reais.

Camada	Responsabilidade nesta distribuição
README	Instalação, opções, saída e referências.
Makefile	Percurso de compilação do executável independente e dos modelos.
src/	Interface C++, física Fortran, modelos incorporados, arquivos paramin e cabeçalhos.
tabs/	Tabelas físicas, EGS4, UrQMD, modelos e leitor de dados comprimidos.
macros/	Exemplos dos autores para leitura e representação. Alguns contêm nomes antigos.
resampler/	Extensões para modificar interações; fora do percurso inicial.
cfg/	Gerado na compilação: configurações param com caminhos substituídos.
obj/Linux/	Objetos intermediários de compilação.
lib/Linux/	Bibliotecas dos modelos e interface.
bin/	Executável conex2r.

> CONCEITO 3.5 / Makefile
>
> Receita de dependências e comandos de compilação lida pelo programa Make. `make sibyll` é um exemplo de compilação; `-m 5` escolhe o modelo durante uma execução posterior.

Para o executável com saída ROOT são necessários compiladores C/C++/Fortran, Make, utilitários Unix, ROOT compatível e desenvolvimento de bzip2 (cabeçalho e biblioteca). O Makefile solicita C++17 e liga `-lbz2`. Python, Jupyter, Conda e Anaconda não são requisitos intrínsecos do CONEX.

`CX_NO_ROOT=1` (uma variável reconhecida pelo Makefile, que fica na pasta principal do CONEX. Você pode passá-la ao comando de compilação, como: `make sibyll CX_NO_ROOT=1`) desliga o percurso ROOT e deixa uma versão de biblioteca para integração. Não é a forma de gerar os arquivos e realizar a instalação analisados neste guia.

> ATENÇÃO /
>
> Um cuidado importante listado em CMakeLists.txt referencia alvos de CORSIKA 8, Sibyll e UrQMD externos: não use `cmake ..` como substituto presumido do Makefile independente.

3.1 Caminhos com espaços

Aspas protegem espaços em Bash, mas o Fortran deste pacote corta certos nomes no primeiro espaço. O Makefile também expande caminhos sem proteção suficiente. Portanto, é uma boa prática manter a *cópia de compilação e as tabelas* em um caminho Linux curto e sem espaços. Evite o uso de caracteres especiais e acentos também.

REFERÊNCIA / Caminhos de arquivos: src/conex2r.F e Makefile.

REFERÊNCIA / Estrutura do pacote: README, Makefile e CMakeLists.txt.

03 / AMBIENTE [continuação]
Guia / APOEMA

Antes de instalar

Linux: escolher um ambiente coerente

OBJETIVO Instalar dependências sem combinar versões incompatíveis por acidente.

> CONCEITO 3.6 / ROOT
>
> Conjunto de ferramentas do CERN para dados científicos: leitura de arquivos, árvores, histogramas e gráficos. Inclui bibliotecas C++ e um interpretador; a interface PyROOT permite usá-lo em Python.

Vamos distinguir duas formas de obter o ROOT: instalar uma distribuição diretamente no sistema ou usar um ambiente Anaconda/Conda. Aqui apresento a primeira apenas como alternativa; em seguida vamos seguir a segunda, que reúne a instalação do ROOT em um comando. Se você está no Windows, prepare antes o Linux pelo WSL 2 e depois retorne a esta página. A principal maneira de instalar o CONEX é no Linux, com ROOT e compiladores de um mesmo ambiente. Pode ser um ambiente já mantido pelo grupo ou uma instalação isolada. Para instalar o ROOT, em Ubuntu/Debian, a receita abaixo é uma **alternativa documentada, no entanto vou mostrar depois outra mais simplificada**: Se escolher a instalação direta no Ubuntu/Debian, execute primeiro as duas linhas com `apt` no terminal Linux. Elas preparam compiladores e bibliotecas; não instalam o ROOT. As linhas seguintes só entram depois de você obter o ROOT pela documentação oficial e substituir `/caminho/para/root` pelo endereço real da instalação.

dependencias do sistema — bash / cód. 3.2

```bash
sudo apt update
sudo apt install build-essential gfortran libbz2-dev
# Instale ROOT conforme a página oficial e sua distribuição.
# Se usar um binário ROOT, selecione sistema, arquitetura e GCC compatíveis.
source /caminho/para/root/bin/thisroot.sh
root-config --version
```

Essa rota fica como referência: a instalação direta do ROOT não será desenvolvida aqui. Para segui-la, consulte as dependências e os binários adequados à sua distribuição, arquitetura e compilador [8, 9]. Se preferir o caminho que vamos desenvolver, continue com Anaconda nas próximas páginas; você não precisa instalar outro ROOT manualmente antes. Nesse caminho, a ativação do Conda prepara o ROOT: não acrescente um `thisroot.sh` de outra instalação.

REFERÊNCIA / ROOT: instalação oficial e dependências [8, 9].

Antes de instalar

Segunda alternativa: Anaconda

OBJETIVO Instalar o gerenciador que vamos usar para obter o ROOT com poucos comandos.

> **CONCEITO 3.7 / Anaconda, Conda e ambiente**
>
> Anaconda é uma distribuição que reúne Python, Conda e ferramentas científicas. Conda instala pacotes e cria ambientes separados: diretórios com suas próprias versões de programas e bibliotecas. Neste caminho, vamos usá-lo para instalar ROOT e PyROOT.

Vamos seguir por Anaconda porque, depois de preparar o ambiente, o Conda obtém o ROOT e suas dependências para nós. Isso simplifica a preparação, especialmente para quem está começando. Python é a linguagem usada pela interface PyROOT; Anaconda é uma conveniência deste percurso, não uma exigência do CONEX.

3.2 Obter e abrir o instalador

1. Acesse anaconda.com/download e procure *Anaconda Distribution*.
2. Selecione o instalador **Linux** para a arquitetura indicada por `uname -a`. No WSL, escolha Linux também: é dentro do Ubuntu que vamos executar o CONEX.
3. Salve o arquivo `.sh`. No gerenciador de arquivos, renomeie-o para `anaconda-installer.sh`, apenas para usarmos um nome curto.
4. Abra um terminal Linux na pasta em que o arquivo foi salvo e execute a linha abaixo. Se baixou pelo navegador do Windows, essa pasta pode estar em `/mnt/c/Users/seu_usuario/Downloads`; confira o nome real antes de entrar nela.

abrir o instalador na pasta do download — bash / cód. 3.3

```bash
bash ./anaconda-installer.sh
```

O instalador apresenta as instruções no próprio terminal. Siga as telas de licença e destino da instalação. Quando ele perguntar se deseja inicializar o Conda na shell, escolha `yes`; isso permite reconhecer `conda` nas próximas sessões. Ao terminar, feche e abra novamente o terminal Linux.

3.3 Conferir a instalação

Agora execute:

conferir o Conda — bash / cód. 3.4

```bash
conda --version
```

O comando deve mostrar uma versão do Conda. Se aparecer "comando não encontrado", confira se reabriu o terminal e se concluiu a inicialização. No destino padrão do instalador, você também pode executar `source "$HOME/anaconda3/etc/profile.d/conda.sh"` na sessão atual; se escolheu outra pasta, use o caminho correspondente.

REFERÊNCIA / Anaconda: instalador Linux; ambientes Conda [14]; ROOT via Conda [8].

03 / AMBIENTE [continuação]
Guia / APOEMA

Antes de instalar

Criar o ambiente e instalar o ROOT

OBJETIVO Executar a preparação em etapas e conferir ROOT e PyROOT no mesmo ambiente.

3.4 1. Criar um ambiente separado

Crie o ambiente conex-apoema no terminal em que o Conda funcionou:

criar o ambiente — bash / cód. 3.5

```bash
conda create -n conex-apoema
```

Confira o resumo e confirme a criação quando solicitado.

3.5 2. Ativar o ambiente

Agora ative o ambiente. A segunda linha configura nele a prioridade estrita de pacotes, para reduzir conflitos entre canais:

ativar e preparar o ambiente — bash / cód. 3.6

```bash
conda activate conex-apoema
conda config --env --set channel_priority strict
```

Confira se aparece (conex-apoema) no prompt e continue nessa sessão.

3.6 3. Instalar o ROOT

Agora instale ROOT pelo canal conda-forge:

instalar ROOT e PyROOT — bash / cód. 3.7

```bash
conda install -c conda-forge root
```

Confirme os pacotes e aguarde o término. Isso instala ROOT e sua interface Python, PyROOT.

3.7 4. Conferir as duas interfaces

Sem sair do ambiente, execute:

conferir ROOT e acesso pelo Python — bash / cód. 3.8

```bash
root-config --version
python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
```

As duas linhas devem mostrar a versão do ROOT; a segunda confirma `import ROOT`. Em cada nova sessão, reative com `conda activate conex-apoema`.

Antes de instalar

Windows e alternativas

OBJETIVO Distinguir Linux no Windows de um porte nativo do CONEX.

3.8 WSL 2: alternativa Windows

Se você usa Windows, vamos preparar o Ubuntu antes de seguir os comandos de Anaconda e CONEX. Faça esta etapa no Windows e, depois, trabalhe no terminal Linux. O WSL 2 executa uma distribuição Linux usando um kernel Linux. Os comandos de instalação do CONEX continuam sendo os de Linux. Em uma máquina sem WSL, a documentação Microsoft orienta abrir PowerShell como administrador e instalar a distribuição; pode ser necessário reiniciar [15]: Abra o PowerShell como administrador e execute `wsl -install -d Ubuntu` se ainda não tiver WSL/Ubuntu. Reinicie se solicitado e conclua a criação do usuário Linux. Depois use as duas últimas linhas para listar as distribuições e abrir o Ubuntu:

PowerShell: receita de preparacao / cód. 3.9

```
wsl --install -d Ubuntu
wsl --list --verbose
wsl -d Ubuntu
```

No Ubuntu, continue o percurso Linux. Um caminho Windows `C:\Users \ana \Downloads` aparece normalmente em `/mnt/c/Users/ana/Downloads`. Para compilar e acessar muitas tabelas, prefira o sistema de arquivos Linux e evite espaços nos caminhos da cópia do CONEX.

3.9 Instalação nativa: viabilidade limitada

ROOT possui distribuição Windows, mas isso não comprova suporte nativo deste pacote. O Makefile reconhece Linux e Darwin; define bibliotecas `.so`/`.dylib`, e a interface usa `dlopen` e `dlfcn.h`. Há referências a `/dev/null`, `/dev/urandom` e ferramentas Unix. Compiladores MinGW não fornecem automaticamente uma combinação compatível com o ROOT para Visual Studio. Um porte exigiria tratar carregamento dinâmico, receitas, interoperabilidade Fortran/C++ e testar a física. Por isso, vamos seguir com o Ubuntu no WSL 2, em vez de tentar uma compilação nativa do CONEX no Windows.

3.10 Outros caminhos pertinentes

Uma máquina virtual Linux é uma alternativa quando WSL não é permitido; exige espaço, memória e instalação própria. Um servidor Linux do grupo também permite compilar e analisar, com transferência apenas dos pequenos resultados. Para um primeiro contato, use Linux ou WSL 2; deixe contêineres e adaptações para macOS para um percurso específico de instalação.

REFERÊNCIA / WSL: documentação Microsoft [15]; distribuição independente: README e Makefile.

04 / INSTALAÇÃO Guia / APOEMA

Instalar o CONEX

Preparar a cópia do CONEX

OBJETIVO Reunir os arquivos e as ferramentas antes de compilar o simulador.

Com ROOT funcionando, vamos preparar o CONEX. Mantenha aberta a sessão com `conex-apoema` ativo. Se escolheu a primeira alternativa de instalação, use a sessão em que ROOT e os compiladores do sistema estejam disponíveis.

4.1 Ferramentas para compilar

No percurso Conda, instale agora as ferramentas de C/C++ e Fortran, Make e bzip2. Elas servem à compilação do CONEX e ficam separadas do passo em que instalamos ROOT. Depois reative o ambiente para carregar a configuração dos compiladores:

ferramentas de compilacao no ambiente ativo — bash / cód. 4.1

```
conda install -c conda-forge cxx-compiler fortran-compiler make bzip2
conda deactivate
conda activate conex-apoema
```

Depois confira se os comandos abaixo mostram versões e um caminho para Make. A ativação dos pacotes define `CXX` e `FC`, que indicam os compiladores:

conferir as ferramentas do ambiente — bash / cód. 4.2

```
"$CXX" --version
"$FC" --version
command -v make
```

Se algum comando falhar, corrija a instalação ou ative novamente o ambiente antes de continuar. Para a primeira alternativa, confira `g++`, `gfortran` e `make` do sistema.

4.2 Organizar os arquivos baixados

Extraia o pacote do CONEX obtido no início da seção 3. Abra a pasta extraída e localize `Makefile`, `README`, `src/` e `tabs/`; essa é a raiz que vamos usar, mesmo que o arquivo baixado tenha criado pastas com nomes repetidos.

Crie, pelo gerenciador de arquivos, a pasta `simulations` dentro da sua pasta pessoal Linux. Copie para ela a raiz completa do CONEX e dê à cópia o nome `conex-7.801`. Isso permite seguir os próximos comandos sem alterar o material baixado. Se usar outro nome, ajuste a variável abaixo.

> **CONCEITO 4.1 / Variável de ambiente e PATH**
>
> Uma variável de ambiente é um valor transmitido pela shell aos programas, como o diretório de saída. PATH é a lista de diretórios onde a shell procura executáveis; não é a lista de tabelas do CONEX.

REFERÊNCIA / Dependências e diretórios: Makefile e README; pacotes de compilação do ambiente Conda.

04 / INSTALAÇÃO [continuação] Guia / APOEMA

Instalar o CONEX

Compilar e conferir a instalação

OBJETIVO Gerar o executável e a biblioteca SIBYLL para a primeira simulação.

Vamos começar por SIBYLL, o modelo que será usado na próxima seção. Assim, você prepara apenas o necessário para a primeira execução. Execute o bloco no terminal Linux com o ambiente ativo:

entrar na copia e compilar SIBYLL — bash / cód. 4.3

```bash
CX="$HOME/simulations/conex-7.801"
cd "$CX"
ls Makefile README src tabs
root-config --version
make sibyll CXX="$CXX" CC="$CC" FC="$FC" > build.log 2>&1
```

`CX` é um atalho criado por nós para o endereço da cópia, não uma opção do CONEX. A linha com `make` compila e guarda as mensagens em `build.log`. Aguarde o retorno do prompt. Se usou os compiladores do sistema na primeira alternativa, substitua apenas essa linha por `make sibyll > build.log 2>&1`.

Agora confira os arquivos gerados e abra a ajuda:

conferir executavel, biblioteca e ajuda — bash / cód. 4.4

```bash
ls -lh bin/conex2r lib/Linux/libCONEXsibyll.so
bin/conex2r -h
```

Você deve encontrar o executável e a biblioteca, e a ajuda deve listar as opções do programa, incluindo o modelo 5, SIBYLL. Se a compilação falhar ou algum arquivo não existir, abra `build.log` e leia o primeiro erro antes de seguir. A ajuda pode encerrar com código não zero mesmo tendo sido exibida; o que importa aqui é conferir seu texto.

`make` sem alvo mostra a ajuda de compilação. `make all` prepara todos os modelos, mas você não precisa disso para a atividade inicial. Concluir a compilação prepara o programa; na seção seguinte vamos conferir sua execução com um chuveiro.

REFERÊNCIA / Makefile: alvos sibyll, binary e dynamic; ajuda do executável.

Instalar o CONEX

Caminhos de configuração e saída

OBJETIVO Saber onde o CONEX procura seus arquivos e onde grava os resultados.

Na próxima seção vamos usar duas variáveis: `CONEX_ROOT`, para indicar os arquivos de configuração, e `ROOT_OUT`, para escolher o diretório de saída. A tabela serve para consulta; você não precisa definir todas essas variáveis para a primeira simulação.

Variável	Para que serve
`CONEX_ROOT`	Diretório dos arquivos param. Se ausente, a interface procura em PWD/cfg.
`ROOT_OUT`	Diretório de saída dos arquivos ROOT. Se ausente, usa PWD; o diretório precisa existir.
`CORSIKA_DATA`	Seleciona a raiz de tabelas durante a compilação, com prioridade sobre CONEXTABROOT.
`CONEXTABROOT`	Alternativa para a raiz de tabelas; o padrão usa tabs da instalação.
`CONEXBINROOT`	Permite escolher outro prefixo de compilação neste Makefile. Para começar, mantenha os arquivos na cópia.
`URQMD13_TAB`	Nome da tabela de larguras de decaimento UrQMD; quando não definida, usa tables.dat.

4.3 Conferir os caminhos antes da primeira execução

Ainda na raiz `CX`, execute:

conferir a configuracao gerada — bash / cód. 4.5

```bash
ls cfg/conex_sibyll.param
head -n 20 cfg/conex_sibyll.param
```

O primeiro comando confirma que o param foi criado; o segundo mostra o começo do arquivo. Confira os caminhos das tabelas: eles devem apontar para a cópia ou para o diretório de dados que você escolheu. Eles foram escritos na compilação; mover o projeto depois pode exigir gerá-los novamente.

`URQMD13_TAB` trata a tabela de larguras de decaimento, `tables.dat`, que pode ser gerada na inicialização. Ela é diferente de `UrQMD-1.3.1-xs.dat`, a tabela de seções de choque indicada por `furqmd dat` no param. Mantenha essa distinção ao diagnosticar arquivos ausentes.

> **ATENÇÃO / Antes de continuar**
>
> Use a sessão em que ROOT está ativo, mantenha `CX` apontando para a cópia compilada e confirme o param. A seção 5 começa a partir dessa preparação. Nas próximas sessões, ative novamente o ambiente e redefina `CX` antes de seguir os exemplos. Evite `make clean` em instalações compartilhadas: trabalhe na sua cópia.

REFERÊNCIA / Makefile; README; interface de execução CxRoot.cc.

05 / PRIMEIRO TESTE Guia / APOEMA

Um chuveiro verificável

Entender uma execução simples

OBJETIVO Entender os parâmetros de uma execução simples e produzir um primeiro resultado verificável.

Uma execução do CONEX começa com a escolha das condições do chuveiro: qual é a partícula primária, qual é sua energia, de que direção ela chega e qual modelo descreve suas interações hadrônicas de alta energia. Também precisamos indicar quantos chuveiros serão simulados.

Vamos começar com uma escolha simples: **um próton de 10^{17} eV, com incidência vertical, usando SIBYLL 2.3e**. Manter essas condições fixas facilita comparar o que pedimos ao programa com o que ele registra no arquivo de saída.

Esta seção apresenta apenas as opções necessárias para esse primeiro exemplo. A seção seguinte detalha as flags, seus valores padrão, outras partículas, os modelos disponíveis e configurações adicionais.

5.1 Como o comando é organizado

O programa é chamado por `bin/conex2r`, seguido das opções que definem a execução. Uma *flag* é uma dessas opções, como `-n`; o valor escrito depois dela informa a escolha, como em `-n 1`.

No primeiro exemplo, vamos usar:

Opção no exemplo	O que estamos escolhendo
`-p 100`	A partícula primária: um próton.
`-m 5`	O modelo hadrônico de alta energia: SIBYLL 2.3e nesta distribuição.
`-n 1`	O número de chuveiros: apenas um.
`-e 17 -E 17`	Os limites inferior e superior da energia, iguais para obter energia fixa.
`-z 0 -Z 0`	Os limites inferior e superior do ângulo zenital, em graus, iguais a zero para incidência vertical.
`-s 314159`	A semente usada pelo gerador pseudoaleatório.
`-x first`	O prefixo que fará parte do nome do arquivo de saída.

5.2 Como informar a energia

As opções `-e` e `-E` recebem o logaritmo decimal da energia em eV:

$$\ell = \log_{10}\left(\frac{E}{\mathrm{eV}}\right), \qquad E = 10^{\ell}\,\mathrm{eV}.$$

Portanto, escrever `-e 17 -E 17` significa usar $E = 10^{17}$ eV. Não significa usar 17 eV. Como os dois limites são iguais, todos os eventos solicitados terão essa energia inicial. Para um núcleo, trata-se da energia total do primário, não da energia por núcleon.

5.3 Como informar um núcleo

Para os primários nucleares considerados aqui, o CONEX usa:

$$\text{código do primário} = 100 \cdot A,$$

em que A é o número de massa, isto é, o número total de prótons e nêutrons do núcleo.

Assim, próton corresponde a 100; hélio-4, a 400; carbono-12, a 1200; e ferro-56, a 5600. Neste primeiro exemplo usamos `-p 100`. Essa regra não deve ser aplicada às partículas não nucleares, que serão discutidas na seção seguinte.

REFERÊNCIA / Distribuição estudada: README e CxRoot.cc, leitura das opções de execução.

05 / PRIMEIRO TESTE [continuação] Guia / APOEMA

Um chuveiro verificável

Executar o primeiro chuveiro

OBJETIVO Executar a simulação e reconhecer os arquivos produzidos.

O bloco de código abaixo realiza sua primeira simulação com o CONEX e organiza os resultados em uma pasta própria. Execute-o no terminal Linux, com ROOT disponível, o ambiente de trabalho ativo e a variável `CX` apontando para a cópia compilada do programa.

> **CONCEITO 5.1 / Seed ou semente**
>
> Valor usado para inicializar o gerador pseudoaleatório, que fornece os sorteios da simulação. Registrar a semente ajuda a reproduzir uma execução, mas repetir seu valor não garante resultados idênticos entre versões, modelos ou ambientes diferentes.

Vamos executar as linhas na ordem apresentada. Se algum comando falhar, pare e verifique o problema antes de continuar. Em particular, se a pasta `first-run` já existir, escolha outro nome e ajuste suas ocorrências no bloco.

primeira execucao bash / cód. 5.1

```bash
cd "$CX"
export CONEX_ROOT="$CX/cfg"
mkdir first-run
export ROOT_OUT="$CX/first-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 314159 -x first > first-run/run.log 2>&1
status=$?
printf 'Exit status: %s\n' "$status"
ls -lh first-run
tail -n 25 first-run/run.log
```

Primeiro, entramos na pasta do CONEX e indicamos onde estão suas configurações com `CONEX_ROOT`. Criamos a pasta `first-run` e a definimos como destino dos arquivos ROOT por meio de `ROOT_OUT`. Essas instruções organizam a execução; não alteram a energia nem o tipo de primário.

A linha iniciada por `bin/conex2r` executa o chuveiro com as escolhas apresentadas na página anterior. A barra invertida ao final da linha apenas permite continuar o mesmo comando na linha seguinte. O trecho `> first-run/run.log 2>&1` guarda as mensagens normais e os erros no mesmo arquivo de texto, chamado `run.log`. Por isso, a simulação pode executar sem mostrar seu progresso no terminal.

Logo após o programa terminar, `status=$?` guarda seu código de saída, e `printf` o mostra na tela. Normalmente, zero indica sucesso. Depois, `ls -lh` lista os arquivos produzidos e seus tamanhos; `tail -n 25` mostra as últimas 25 linhas do log para conferir como a execução terminou.

> **ATENÇÃO / Evitar sobrescrita**
>
> A variável `ROOT_OUT` não cria uma pasta: por isso usamos `mkdir` antes da simulação. Além disso, repetir prefixo, modelo, semente e partícula no mesmo diretório pode substituir um arquivo anterior, mesmo que a energia ou o ângulo tenham mudado. Use uma pasta nova para cada execução que deseja preservar.

REFERÊNCIA / CxRoot.cc: leitura das opções, `ROOT_OUT` e construção do nome de saída.

05 / PRIMEIRO TESTE [continuação] Guia / APOEMA

Um chuveiro verificável

Conferir o que foi produzido

OBJETIVO Verificar se o arquivo abre e se suas informações correspondem ao comando executado.

Com as escolhas do exemplo, o nome esperado nesta distribuição é:

`first_sibyll23e_000314159_100.root`

Esse nome reúne o prefixo `first`, a identificação do modelo `sibyll23e`, a semente preenchida com zeros e o código do primário 100. Confira o nome efetivamente produzido em sua pasta antes de abrir o arquivo. O link no nome leva à receita em `results`; esse arquivo deve ser gerado localmente, pois os eventos não são distribuídos no repositório.

5.4 O que caracteriza uma execução bem-sucedida

Não basta encontrar um arquivo com extensão `.root`. Ele pode ter sido criado antes de uma interrupção. Vamos conferir três pontos:

1. O log termina sem indicar falha de configuração, biblioteca ou tabela.
2. O arquivo abre no ROOT e contém as árvores `Header` e `Shower`.
3. A árvore `Shower` contém uma entrada, e as informações gravadas correspondem ao próton, à energia e à direção solicitados.

Uma árvore ROOT organiza os dados em entradas e campos. Por enquanto, basta entender que `Header` reúne informações da execução e `Shower` contém os dados dos chuveiros. Os objetos ROOT e suas estruturas serão estudados com mais cuidado na seção de análise.

05 / PRIMEIRO TESTE [continuação] Guia / APOEMA

Um chuveiro verificável

Abrir o arquivo e gerar o perfil

OBJETIVO Conferir a estrutura da saída e produzir os primeiros gráficos.

Vamos usar Python com PyROOT para esta primeira conferência, aproveitando o ambiente Conda preparado na instalação. Na seção 7 esse caminho será explicado passo a passo. Se preferir C++, a seção 8 ensina as mesmas operações com macros: você só precisa seguir uma das duas rotas.

Baixe o repositório como explicado na página 4 e abra o terminal Linux na pasta extraída, com o ambiente ativo e `CX` definido. O script `python/conex.py` está disponível no link; baixe o pacote completo para conservar as pastas. Execute:

inspecionar e desenhar o primeiro perfil — bash / cód. 5.2

```bash
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
python python/conex.py inspect "$ARQ"
python python/conex.py profile "$ARQ" --output profile-python
```

No GitHub: `python/conex.py`; receita dos dados (primeiro).

A variável `ARQ` guarda o caminho absoluto do arquivo gerado; ajuste-a se usou outra organização. O primeiro comando de análise lista as chaves, as árvores, seus tipos e a primeira entrada. Uma mensagem de ausência de `LeadingInteractions` é normal neste teste, que não pediu `-K`.

O segundo comando seleciona o evento zero, a primeira entrada, e cria `profile-python.pdf` e `profile-python.csv` na pasta atual. O PDF reúne os dois gráficos; o CSV guarda os valores usados e deixa o último ponto sem intervalo de depósito associado. Use um prefixo novo se esses arquivos já existirem: o script recusa sobrescrita.

> **RESULTADO 5.1 / O que procurar no primeiro arquivo**
>
> Para esta execução, esperamos uma entrada em `Header` e uma em `Shower`. Confira o primário 100, o modelo de alta energia 5, a energia logarítmica 17 e o ângulo zenital zero. O perfil longitudinal pode variar entre realizações; sua forma exata não é um critério de sucesso da instalação.

REFERÊNCIA / `python/conex.py`: inspect e profile; estrutura de saída definida em CxRoot.cc.

05 / PRIMEIRO TESTE [continuação] Guia / APOEMA

Um chuveiro verificável

Interpretar o primeiro perfil

OBJETIVO Relacionar os gráficos ao desenvolvimento do chuveiro, sem tirar conclusões estatísticas de um único evento.

Abra o arquivo `profile-python.pdf` que acabou de gerar. Vamos observar como a cascata se desenvolve ao atravessar a atmosfera. A figura abaixo apresenta uma realização de referência com as mesmas escolhas do exemplo: próton, 10^{17} eV, incidência vertical e SIBYLL 2.3e.

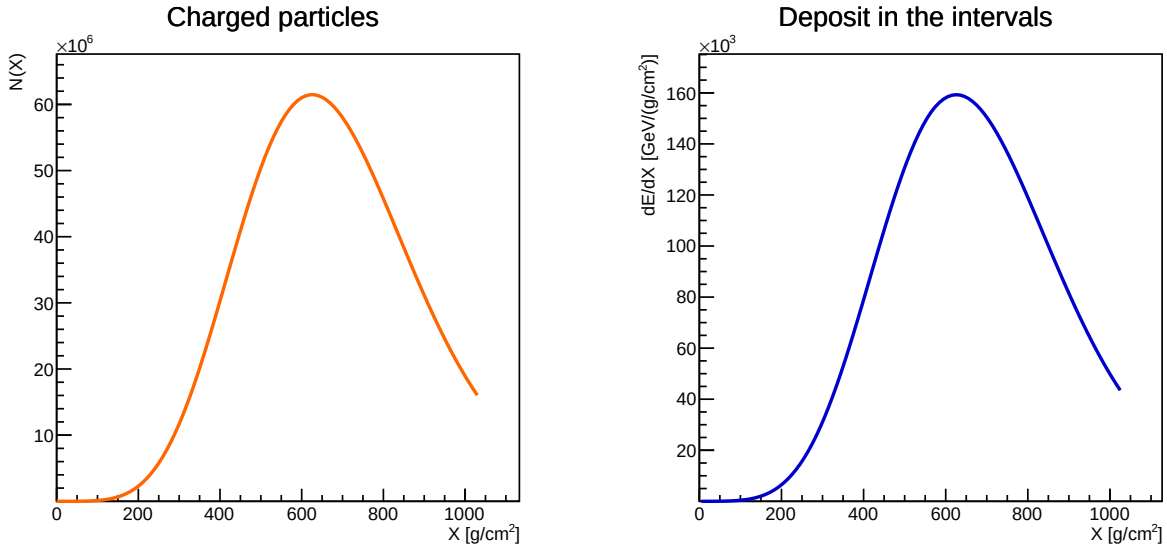


Figura 5.1: Exemplo real de um chuveiro iniciado por próton de 10^{17} eV, vertical, com SIBYLL 2.3e e UrQMD, semente 314159. Figura gerada pela macro `profile.C`. Os dois gráficos têm grandezas diferentes no eixo vertical.

5.5 O que estamos observando

Nos dois gráficos, o eixo horizontal representa a profundidade atmosférica X, em $\mathrm{g\,cm^{-2}}$, ao longo do desenvolvimento do chuveiro. Ela representa a quantidade de matéria atravessada por unidade de área, não a altitude.

O gráfico de partículas carregadas mostra como seu número varia com a profundidade. No início, as interações produzem secundárias e a população cresce. Ela atinge um máximo e depois diminui, à medida que a energia é repartida e as partículas deixam de sustentar a multiplicação da cascata.

O outro gráfico mostra o depósito de energia por unidade de profundidade, dE/dX, em $\mathrm{GeV/(g\,cm^{-2})}$. Ele descreve uma grandeza diferente da contagem de partículas; por isso, os dois máximos não precisam ocorrer exatamente na mesma profundidade.

05 / PRIMEIRO TESTE [continuação] Guia / APOEMA

Um chuveiro verificável

Um primeiro resultado resolvido

No evento de referência, a soma do depósito nos 103 intervalos entre os 104 pontos armazenados é aproximadamente $7{,}98 \times 10^{7}$ GeV. Comparando com a energia inicial,

$$10^{17}\,\mathrm{eV} = 10^{8}\,\mathrm{GeV},$$

essa soma corresponde a cerca de 80% da energia do primário.

Esse valor é a integral do depósito nos intervalos registrados pela macro. Não representa, por si só, a energia medida por um detector nem demonstra que toda a energia inicial foi depositada nesse trecho da atmosfera.

Observe também se o perfil cresce e depois diminui dentro do intervalo apresentado. Um máximo fora do trecho registrado ou um perfil interrompido exige conferir a geometria, o solo e os limites de profundidade antes de interpretar o resultado.

> **ATENÇÃO / O alcance deste primeiro exemplo**
>
> Um único chuveiro permite aprender o fluxo de execução e leitura, mas não determina uma média nem uma distribuição de flutuações. Diferenças entre dois eventos isolados não bastam para concluir que um primário ou modelo produz sistematicamente determinado comportamento.

EXERCÍCIO 5.1 / Para praticar, sem gabarito

1. Confira no arquivo se o primário, a energia e a direção correspondem ao comando. Explique por que usamos dois limites iguais para a energia e dois para o ângulo.
2. Execute um novo chuveiro com outra semente, em uma pasta diferente, e compare os perfis. Descreva uma semelhança e uma diferença sem atribuir significado estatístico a elas.
3. Usando a regra $100A$, escreva o código do ferro-56. Identifique qual opção do comando precisaria mudar para usá-lo e explique como preservaria os arquivos da execução anterior.

REFERÊNCIA / Figura e valores do exemplo: `results/PROVENANCE.md`. As duas rotas calculam a integral nos mesmos intervalos armazenados.

06 / CONFIGURAÇÃO Guia / APOEMA

Energia e amostragem

O que é um argumento de execução

OBJETIVO Prever as condições de um conjunto antes de rodar o programa.

Uma flag é uma opção do executável, como `-n 12`. Ela não é uma opção do compilador. Já `CX_NO_ROOT=1` modifica a compilação; `ROOT_OUT` é uma variável de ambiente; `set fehcut 0.005` é uma instrução no arquivo físico de configuração. Esses quatro mecanismos têm momentos e efeitos diferentes.

Sintaxe	Padrão no código	Significado, restrições e exemplo
`-e valor`	16.5	Limite inferior em $\log_{10}(E/\mathrm{eV})$. Ex.: `-e 17`.
`-E valor`	21	Limite superior na mesma unidade. Ex.: `-E 17`.
`-a valor`	3	Índice de amostragem, adimensional. Ex.: `-a 0`.
`-n inteiro`	1	Número de chuveiros. Use inteiro positivo. Ex.: `-n 12`.

6.1 A distribuição de energias importa

Se $\alpha > 1$, com α definido como a opção -a valor, `GetEnergy` sorteia uma lei $dN/dE \propto E^{-\alpha}$ e rejeita valores acima do máximo. Se $0 \leq \alpha \leq 1$, sorteia uniformemente em $\log_{10} E$, o que corresponde a $dN/dE \propto 1/E$. Se $\alpha < 0$, sorteia em uma grade de passo $|\alpha|$ no logaritmo; não percorre a grade em sequência ordenada. Se os limites são iguais, o índice não altera a energia.

> **EXEMPLO 6.1 / Duas maneiras diferentes de preencher um intervalo**
>
> `-e 17 -E 18 -a 0` distribui eventos igualmente por intervalo de logaritmo. `-e 17 -E 18 -a 3` favorece as energias menores segundo uma lei em energia linear. Os histogramas não respondem à mesma pergunta sem considerar a seleção e, quando pertinente, pesos.

6.2 Entradas inválidas não são uma interface protegida

O parser usa `atoi`/`atof`, com verificação limitada. Um texto malformado pode virar zero. Use ponto decimal e confira o resumo impresso. Exija $e \leq E$ antes da chamada: limites invertidos podem impedir o término da rejeição de energia. Não inferimos faixas físicas só da aceitação pelo parser. A descrição interna cita $0{,}1$ GeV a 10^{13} GeV, mas os modelos e tabelas também limitam a aplicabilidade. Os testes deste guia não validam os extremos.

REFERÊNCIA / CxRoot.cc: construtor, GetOptions:660–755 e GetEnergy:510–539; README, comentário de ConexInit no Fortran.

06 / CONFIGURAÇÃO [continuação] Guia / APOEMA

Energia e amostragem

Direção e geometria

OBJETIVO Distinguir direção fixa de distribuição de incidência.

Sintaxe	Padrão	Unidade, significado e exemplo
`-z valor`	60	Menor zenital, graus. `-z 0`.
`-Z valor`	60	Maior zenital, graus. `-Z 0`.
`-i valor`	0	Azimute fixo em graus; negativo sorteia em $[0, 360)$. `-i -1`.
`-o valor`	0	Menor parâmetro de impacto em metros. `-o 0`.
`-O valor`	0	Maior parâmetro de impacto em metros. `-O 0`.

6.3 Amostragem angular

O README documenta $0 \leq \theta \leq 180°$: acima de $90°$ o chuveiro é ascendente. Para limites diferentes, o código implementa a incidência isotrópica sobre superfície plana, com distribuição proporcional a $|\cos\theta|$ na variável $\cos\theta$ em um mesmo hemisfério. Isso não é uniforme em θ. A direção fixa é obtida igualando os limites; para horizontal, fixe ambos em $90°$.

O código interrompe um intervalo com limite inferior abaixo e superior acima de $90°$. Não misture hemisférios e não use faixas invertidas. Os testes que realizamos são verticais; casos horizontais e ascendentes ficam como opções documentadas e lidas no código.

6.4 Azimute: convenção explícita

A ajuda e o README especificam a convenção Auger, $0°$ a leste. O código usa projeções $x \propto \cos\phi$ e $y \propto \sin\phi$. O material anterior dizia "sentido horário", mas não apresentava a orientação dos eixos necessária para sustentar isso. Este guia não repete esse sentido como fato: confira o sistema de coordenadas antes de relacionar o azimute a uma imagem ou mapa. Para o exemplo vertical, mantemos `-i 0`.

6.5 Parâmetro de impacto e solo

`GetImpactParameter` sorteia linearmente entre `-o` e `-O`. Esse valor chega ao Fortran como `altitude`, altura do ponto de maior aproximação ao centro da Terra em relação a `hground`; não é uma distância lateral de detector. `hground` é a altitude do observador no param. Não altere os três valores simultaneamente no primeiro teste. O branch `altitude/D` registra o valor passado, em metros.

REFERÊNCIA / CxRoot.cc:GetTheta, GetPhi e GetImpactParameter; conex2r.F:RunConex e conex_sub.F:Initialize2.

Energia e amostragem

Primários e modelos

OBJETIVO Usar códigos válidos sem confundir convenções de partículas.

Opção	Padrão	Valores relevantes e exemplos
`-p inteiro`	100	Código de primário: próton 100, He-4 400, C-12 1200, Fe-56 5600, fóton 0 ou 22, elétron 11, pósitron -11.
`-m inteiro`	4	Modelo de alta energia entre os modelos efetivamente compilados e com tabelas disponíveis.
`-H inteiro`	0	1 remove a componente eletromagnética. Use 0 ou 1; o parser transforma qualquer inteiro não zero em 1.

Núcleos usam $100 \cdot A$, com número de massa A, não o esquema PDG de íons. Os demais códigos seguem as partículas não nucleares aceitas pela implementação; isso não habilita qualquer partícula PDG. Para transformar nosso exemplo de próton em fóton, você troca `-p` pelo código PDG de uma partícula aceita pelo CONEX. Basta substituir, então, `-p 100` por `-p 22`, mantendo as demais escolhas. O README exclui chuveiros iniciados por neutrinos e taus.

Tabela 6.1: Correspondência verificada no README, lista de modelos no código-fonte e ajuda gerada pelo código.

Código	Modelo	Alvo Make	Parte do nome de saída
2	QGSJET01	qgsjet	qgsjet01
4	EPOS LHC-R	epos	eposlhcr
5	SIBYLL 2.3e	sibyll	sibyll23e
6	QGSJETII-04	qgsjetII	qgsjetII04
9	DPMJET III.2017-1	dpmjetIII	dpmjetIII
10	QGSJETIII-01	qgsjetIII	qgsjetIII

UrQMD é o modelo de baixa energia nos param fornecidos, não uma alternativa válida para `-m`. A interface de parser deixa passar alguns números sem modelo correspondente; a lista da Tabela 6.1, a biblioteca e o arquivo de configuração devem concordar.

O limiar entre *modelos hadrônicos* de alta e baixa energia é diferente do limiar *Monte Carlo/equações de cascata*. O README informa 80 GeV para os modelos e 30 GeV para EPOS LHC-R. Confira `HiLowEgy` no Header do run; para SIBYLL o teste verifica o valor efetivo. Não use a frase antiga "MC acima de 80 GeV" como definição do método híbrido.

REFERÊNCIA / README; conexHEModels.h; CxRoot.cc:GetOptions e rootOut; arquivos paramin dos modelos.

06 / CONFIGURAÇÃO [continuação] Guia / APOEMA

Energia e amostragem

Sementes, salvamento e detalhes

OBJETIVO Controlar nomes e reprodução sem depender de defaults silenciosos.

Sintaxe	Padrão	Significado, exemplo e armadilha
`-s inteiro`	0	Seed. Use positivo, por exemplo `-s 314159`. Zero solicita leitura de /dev/urandom no código atual.
`-S inteiro`	10	Intervalo de autosave em eventos. Ex.: `-S 2`. Deve ser positivo: zero aparece como divisor de módulo no código.
`-x texto`	conex	Prefixo do nome. Ex.: `-x proton`. Não cria diretório e não é o nome completo do arquivo.
`-K inteiro`	0	Quantas interações sucessivas da cadeia da partícula líder registrar. Ex.: `-K 1`. Limitado pela capacidade de arrays de 100 interações; use bem abaixo disso.
`-h`	–	Ajuda com opções compiladas; não inicia simulação.

6.6 O que a semente garante e o que não garante

O autosave merece uma ressalva de código: a checagem usa `GetEntries()+1` depois de `Fill`. Para $S > 1$, a primeira chamada ocorre após $S - 1$ entradas, e depois em intervalos de S. Não é uma garantia literal de checkpoint exatamente após o evento S. A gravação final permanece essencial.

O README afirma que $s \leq 0$ gera uma semente automaticamente, com possíveis alternativas de data e valor 123. O C++ desta cópia só trata explicitamente $s = 0$, lendo `/dev/urandom`. Evite negativos e forneça um inteiro positivo pequeno. Não extrapole o tamanho de inteiro do parser.

O README também adverte que reprodução pela mesma seed estaria limitada ao QGSJet por conversão de precisão. A advertência deve ser preservada mesmo que um teste local de outro modelo coincida. O registro de testes desta edição compara realizações de SIBYLL no mesmo ambiente; não afirma equivalência binária entre plataformas ou versões. Arquivos ROOT podem conter metadados diferentes mesmo quando as grandezas dos eventos coincidem.

6.7 Interações líderes

`-K 1` salva secundárias da primeira interação; `-K 2` acrescenta as secundárias da próxima interação da partícula líder mais energética, e assim por diante. Não salva todas as ramificações nem gerações inteiras da árvore genealógica. A árvore se chama `LeadingInteractions`, quando habilitada na compilação e solicitada com $K > 0$. O padrão de execução é `-K 0`, sem gravar essa árvore. As duas rotas de análise apresentam suas branches e como lê-las: Python ou C++. Os cabeçalhos desta cópia a habilitam, inclusive momentos e códigos de saída CORSIKA; esses códigos não são automaticamente os mesmos aceitos por `-p`.

6.8 Extensões avançadas

`-X,-P,-R,-M,-T,-L,-F` dependem de `CONEX_EXTENSIONS`. Não fazem parte da compilação padrão testada. São ferramentas de modificação da física, não ajustes cosméticos. A referência científica indicada é Ulrich et al. [4]; leia-a antes de ativar essas opções. A ajuda compilada é a referência para disponibilidade.

REFERÊNCIA / Código: CxRoot.cc:767–773, rootOut e autosave; conexConfig.h; README: comentários de -s e -K.

06 / CONFIGURAÇÃO [continuação] Guia / APOEMA

Energia e amostragem

Arquivos físicos e tabelas

OBJETIVO Alterar um parâmetro de modo consciente e manter rastreabilidade.

> **CONCEITO 6.1 / Arquivo de configuração**
>
> Texto lido pelo programa para determinar caminhos e parâmetros físicos. Aqui os modelos têm arquivos `conex_*.param`, gerados de `src/*.paramin` pelo Makefile. Comentários começam com `!`.

A flag `-m 5` escolhe a biblioteca e `conex_sibyll.param`; dentro dele `model sibyll` e `lemodel urqmd` devem concordar com a escolha. Energia, direção e número de eventos são controlados pela interface de execução. Não adicione instruções de outro modo do CONEX supondo que elas prevalecerão sobre as flags.

Entrada em SIBYLL	Valor	Interpretação e limite verificável
`set ixmax`	1	Ajuste Gaisser–Hillas ligado. Zero desliga o ajuste, não o cálculo do perfil.
`set hground`	0	Altitude do solo em metros.
`set fehcut`	0.005	Fração de transição MC/CE para hádrons.
`set feecut`	0.005	Fração para componente eletromagnética.
`set femcut`	0.0005	Fração para múons.
`set hacut1`	1	GeV, corte do perfil principal hadrônico/muônico.
`set hacut2/3`	1	GeV, perfis de múons/hádrons; cada um $\geq$ hacut1.
`set emcut1`	0.001	GeV, corte eletromagnético principal (1 MeV).
`set emcut2/3`	0.001	GeV, fótons/elétrons; cada um $\geq$ emcut1.

Os cortes especificam quais partículas entram em cada perfil; não são limiares MC/CE. Os comentários recomendam pelo menos 0,3 GeV para o corte hadrônico e 1 MeV para eletromagnéticas. Os limiares relativos são aplicados em `ConexRun`, com dependência da energia por núcleon para núcleos e limites internos de tabelas; não se resumem a "uma energia fixa de 80 GeV".

`xmaxp` limita a profundidade simulada; se não definido pelo usuário, o programa evolui até o solo na geometria escolhida. O README menciona máximo técnico de $37000\,\mathrm{g\,cm^{-2}}$. Uma mudança de solo ou profundidade pode truncar o máximo. Não ajuste cortes e limiares para "acelerar" sem estudar o efeito no observável.

> **EXERCÍCIO 6.1 / Para praticar, sem gabarito**
>
> 1. Monte um comando para energia fixa e explique por que são necessárias as duas flags de energia.
> 2. Explique a diferença entre hacut2 e femcut sem executar alterações.
> 3. Compare -K 2 com a ideia de salvar duas gerações completas: por que são diferentes?

REFERÊNCIA / src/conex_sibyll.paramin; src/conex_sub.F:ConexRun; README: steering files.

06 / CONFIGURAÇÃO [continuação] Guia / APOEMA

Energia e amostragem

Como conferir os caminhos das tabelas

OBJETIVO Diagnosticar uma tabela ausente antes de iniciar uma campanha.

Nos param, `fconex`, `fegs` e `furqmd` associam funções a arquivos. A compilação substitui `@CONEXTABROOT@` e `@BZ2@`. Os caminhos ficam gravados, geralmente absolutos. Um pacote movido pode continuar apontando para o local anterior.

verificacao antes do run bash / cód. 6.1

```bash
cd "$CX"
printf '%s\n' "$CONEX_ROOT" "${CORSIKA_DATA:-not set}"
head -n 30 cfg/conex_sibyll.param
ls -lh tabs/CONEX/conex.wsib20.bz2
ls -lh tabs/CONEX/conex.wurq20.bz2
ls -lh tabs/UrQMD/UrQMD-1.3.1-xs.dat
ldd lib/Linux/libCONEXsibyll.so
```

6.9 O que falta no pacote e quando isso importa

O inventário local contém 170 arquivos, cerca de 79 MiB, sem binários prontos. Inclui fontes e tabelas do modo 1D para SIBYLL, mas não o conjunto volumoso QGSJetII/QGSJetIII. O param SIBYLL também referencia tabelas `p2dec`, `p2urq` e `p2sib` ausentes localmente; no código essas leituras ficam condicionadas aos modos de momentos/3D. Uma verificação que exija todos os caminhos de todos os param pode acusar itens que não entram no modo 1D testado. Não remova entradas aleatoriamente para silenciar mensagens.

O README aponta o repositório institucional `AirShowerPhysics/corsika-data` no GitLab do KIT [6]. Para outro modelo, obtenha as tabelas compatíveis, verifique o conteúdo e selecione a raiz antes de gerar os param. QGSJETIII possui ainda instrução de descompressão específica no README. Não copie um endereço de GitHub antigo como se fosse o repositório indicado pela distribuição atual.

6.10 Compressão: divergência investigada

O README sugere Boost e `NO_BOOST=1`. Este Makefile usa `Interface.bz2.cc`, inclui `bzstream.hpp`, liga `-lbz2` e não implementa uma seleção `NO_BOOST`. A compilação testada usa bzip2. Se optar por descomprimir arquivos manualmente, os nomes nos param precisam acompanhar a mudança; o simples uso da variável antiga não troca o leitor desta cópia.

> **ATENÇÃO / Dados externos e redistribuição**
>
> O pacote entregue contém pequenos resultados próprios, não o CONEX nem tabelas dos modelos. Não se presume uma licença única a partir de um componente: o leitor declara GPLv3 e UrQMD tem aviso próprio; o pacote local não traz uma licença geral inequívoca. Obtenha a distribuição pelos canais dos autores e preserve suas condições e citações.

REFERÊNCIA / Inventário local; condicionais do Fortran junto às leituras p2; Makefile e tabs/readLib; [6, 7].

07 / ANÁLISE EM PYTHON Guia / APOEMA

Ler os resultados com PyROOT

Escolha uma das duas rotas

OBJETIVO Abrir a saída, reconhecer sua estrutura e analisar os dados usando Python.

Vamos propor duas maneiras de ler os arquivos: Python com PyROOT nesta seção ou macros C++ na seção 8. As duas cobrem as mesmas árvores, grandezas e operações. **Basta seguir uma delas** e depois continuar em Uso cotidiano. Escolha esta rota se prefere Python.

PyROOT é a interface Python do ROOT, instalada com o pacote ROOT do percurso Conda. Vamos usar `import ROOT` e a biblioteca padrão do Python. No terminal Linux, ative o ambiente e confira o acesso:

preparar a sessao Python — bash / cód. 7.1

```
conda activate conex-apoema
python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
python
```

O caminho em `ARQ` pressupõe que `CX` foi definido como na instalação. Ajuste-o se organizou a simulação em outra pasta. O último comando abre o interpretador Python. A partir daí, os blocos identificados como Python são digitados nele; não são comandos Bash.

> **CONCEITO 7.1 / Árvore e branch**
>
> Uma TTree organiza entradas (aqui, eventos ou metadados). Branch é uma coluna ou grupo de valores da árvore. Uma entrada pode conter um escalar ou um vetor: um perfil inteiro pertence a um evento.

Agora abra o arquivo somente para leitura e descubra o que ele contém:

abrir e inspecionar o arquivo — Python / cód. 7.2

```
import os
import ROOT
f = ROOT.TFile.Open(os.environ["ARQ"], "READ")
if not f or f.IsZombie():
    raise RuntimeError("Cannot open ROOT file")

f.ls()
h = f.Get("Header")
s = f.Get("Shower")
if not h or not s:
    raise RuntimeError("Header or Shower is missing")

print(h.GetEntries(), s.GetEntries())
s.Print()
h.Show(0)
```

`f.ls()` lista as chaves; `Get` recupera uma árvore; `GetEntries` conta entradas; `Print` mostra sua estrutura; `Show(0)` mostra a primeira entrada. No primeiro exemplo esperamos uma entrada em cada árvore. Mantenha `f` aberto enquanto usa suas árvores.

REFERÊNCIA / ROOT: interface Python [16], TFile e TTree [11, 10]; `python/conex.py`.

07 / ANÁLISE EM PYTHON [continuação] Guia / APOEMA

Ler os resultados com PyROOT

Ler os tipos e as dimensões

OBJETIVO Selecionar o observável certo e evitar buffers de tamanho incorreto.

Tabela 7.1: Parte do esquema implementado na saída 2.51 em modo 1D.

Árvore / branch	Tipo ROOT	Significado
Header / Seed1, Particle	Int_t	Semente inicial e código do primário.
Header / lgEmin, lgEmax	Double_t	Limites de $\log_{10}(E/\mathrm{eV})$.
Header / Version	Float_t	Identificação gravada pelo programa; distinta de OutputVersion.
Header / OutputVersion	Float_t	Versão de formato, independente da anterior.
Header / HEModel, LEModel	Int_t	Códigos dos modelos efetivos.
Header / HiLowEgy	Float_t	Transição dos modelos hadrônicos, GeV.
Shower / lgE, zenith, azimuth	Float_t	Energia logarítmica e ângulos em graus.
Shower / altitude	Double_t	Parâmetro de impacto em metros.
Shower / nX	Int_t	Número de pontos de cada perfil.
Shower / X, H, D	Float_t[nX]	Profundidade, altitude e distância ao longo do eixo.
Shower / N, Mu, Electrons	Float_t[nX]	Populações acima dos cortes relevantes.
Shower / Gamma, Hadrons	Float_t[nX]	Fótons e hádrons dos perfis próprios.
Shower / dEdX, dMu	Float_t[nX]	Depósito por profundidade e produção de múons.
Shower / EGround	Float_t[3]	Energia em GeV: e+γ, hádrons, múons no limite final.

`Float_t` e `Double_t` são números de precisão simples e dupla; `Int_t` é inteiro. `/F`, `/D` e `/I` nas leaflists indicam esses tipos. Não ligue um endereço de `double` a um branch `/F`. `TTreeReaderArray<Float_t>` lê arrays de comprimento variável e evita assumir um buffer de 5000 ou 20000 pontos.

7.1 Outros campos que exigem cuidado

`Xfirst` é a profundidade da primeira interação em $\mathrm{g\,cm^{-2}}$, `Hfirst` é sua altitude em metros, e `XfirstIn` é inelasticidade, não profundidade apesar do nome. `Seed2` e `Seed3` guardam contadores de estado do gerador, não dois novos números escolhidos pelo usuário. `cpuTime` está em segundos de CPU por evento e não inclui necessariamente todo o custo de inicialização. `SVNRevision`, ainda listado no README, não é criado na árvore pelo C++ atual.

REFERÊNCIA / CxRoot.cc:923–944 e 1042–1077; README; registros/esquema-root.txt quando gerado no teste.

7.2 Conferir a folha antes de copiar o vetor

Ainda no Python, com `f` e `s` abertos, execute:

tipo e dimensao do perfil Python / cód. 7.3

```python
folha = s.GetBranch("X").GetListOfLeaves().At(0)
print(folha.GetTypeName())
print(folha.GetLeafCount().GetName())
s.GetEntry(0)
print(int(s.nX), float(s.lgE), float(s.zenith))
```

Para `X`, a saída estudada informa `Float_t` e o contador `nX`. `GetEntry(0)` carrega os valores do primeiro evento. Os vetores são atualizados ao carregar outra entrada: copie seus valores se quiser conservá-los.

07 / ANÁLISE EM PYTHON [continuação]
Guia / APOEMA

Ler os resultados com PyROOT

Representar o perfil com Python

OBJETIVO Copiar os vetores de um evento e gerar gráficos com unidades corretas.

Vamos separar a profundidade e o número de partículas do primeiro evento. No Python em que o arquivo continua aberto, execute:

copiar os valores de um evento — Python / cód. 7.4

```
s.GetEntry(0)
n = int(s.nX)
x = [float(s.X[i]) for i in range(n)]
ncar = [float(s.N[i]) for i in range(n)]
dep = [float(s.dEdX[i]) for i in range(n - 1)]
meio = [(x[i] + x[i+1])/2 for i in range(n - 1)]
integral = sum(dep[i]*(x[i+1]-x[i]) for i in range(n-1))
print("Deposit over intervals [GeV]:", integral)
```

Há `nX` planos para `X` e `N`, mas apenas `nX-1` intervalos entre eles. O depósito `dEdX[i]` representa o intervalo entre `X[i]` e `X[i+1]`; por isso usamos sua posição média, sem extrapolar o último ponto. A soma calcula o depósito nos intervalos registrados, não a energia medida por um detector.

Um `TGraph` desenha pares ordenados; `TCanvas` é a superfície de desenho. Para salvar o perfil de partículas carregadas, continue:

desenhar e exportar sem bibliotecas adicionais — Python / cód. 7.5

```
from array import array
ROOT.gROOT.SetBatch(True)
c = ROOT.TCanvas("c", "Profile", 800, 500)
g = ROOT.TGraph(n, array("d", x), array("d", ncar))
g.SetTitle("Charged particles;X [g/cm^{2}];N(X)")
g.Draw("AL")
c.Print("charged-profile.pdf")
f.Close()
exit()
```

Isso cria um PDF na pasta de trabalho; `exit()` retorna ao terminal. Para gerar os dois gráficos, o CSV e a integral com verificações de tipos, dimensões e valores finitos, use o script pronto. Abra o terminal na raiz do repositório baixado e execute:

perfil completo com PyROOT — bash / cód. 7.6

```
python python/conex.py profile "$ARQ" --output profile-study
```

No GitHub: `python/conex.py`.

Serão criados `profile-study.pdf` e `profile-study.csv`; o novo prefixo preserva a saída da seção 5. Ao repetir, escolha outro prefixo: o script recusa sobrescrita. Compare as unidades e a forma das curvas com o primeiro perfil.

REFERÊNCIA / `python/conex.py`: read_profile e profile; README: posição de dEdX; ROOT TGraph [17].

07 / ANÁLISE EM PYTHON [continuação]
Guia / APOEMA

Ler os resultados com PyROOT

Histogramas e resumos com Python

OBJETIVO Distinguir um perfil de um evento da distribuição de máximos de vários eventos.

> CONCEITO 7.2 / Histograma
>
> Agrupa valores em intervalos, chamados bins. A altura pode ser contagem, fração ou densidade; diga qual é usada. No ROOT os bins normais começam em 1; o bin 0 é underflow e o bin final extra é overflow.

Para cada evento, `XmxdEdX` fornece um máximo do depósito por interpolação quadrática. Um histograma dessa branch reúne **um valor por chuveiro**; não devemos preenchê-lo com todos os valores do vetor `X`.

Vamos produzir uma amostra didática de 12 prótons e 12 ferros, a 10^{17} eV, verticais, com sementes distintas e os mesmos modelos e configurações. Na raiz do repositório baixado, com o ambiente ativo e `CX` definido, execute:

preparar uma pequena amostra — bash / cód. 7.7

```bash
bash scripts/campaign.sh "$CX" "$PWD/my-campaign"
export A="$PWD/my-campaign/proton_sibyll23e_000271828_100.root"
export B="$PWD/my-campaign/iron_sibyll23e_000271829_5600.root"
python
```

O destino deve ser novo. O `script` guarda parâmetros, configurações e logs. Se já produziu essa campanha, pule a primeira linha e defina `A` e `B` com seus arquivos existentes. No Python, vamos desenhar a distribuição de prótons:

um valor por evento no histograma — Python / cód. 7.8

```python
import os, ROOT
ROOT.gROOT.SetBatch(True)
f = ROOT.TFile.Open(os.environ["A"], "READ")
s = f.Get("Shower")
hX = ROOT.TH1D("hX", ";Xmax dE/dX [g/cm^{2}];Eventos", 30, 0, 1200)
s.Draw("XmxdEdX>>hX", "", "goff")
hX.SetDirectory(0)
print(hX.GetEntries(), hX.GetBinContent(0), hX.GetBinContent(31))
c = ROOT.TCanvas("cHist", "Maxima", 800, 500)
hX.Draw("HIST")
c.Print("interactive-histogram.pdf")
f.Close()
exit()
```

São 30 bins de $40\,\mathrm{g\,cm^{-2}}$. Confira as contagens fora da faixa: o bin 0 é underflow e o 31 é overflow. Para um resumo com seleção de valores finitos e positivos, execute no terminal:

histograma e resumo do conjunto — bash / cód. 7.9

```bash
python python/conex.py histogram "$A" --output histogram-python
```

O PDF mostra as contagens; o CSV reúne entradas válidas, rejeitadas, média, desvio, erro da média e bins externos. O resumo exige dois eventos válidos e inclui os máximos aceitos fora da faixa desenhada.

REFERÊNCIA / `python/conex.py`: maxima, summary e histogram; ROOT TH1 [12]; `scripts/campaign.sh`.

07 / ANÁLISE EM PYTHON [continuação] Guia / APOEMA

Ler os resultados com PyROOT

Comparar conjuntos com Python

OBJETIVO Comparar duas amostras mantendo fixas as condições que não fazem parte da pergunta.

Vamos comparar os máximos do depósito dos dois conjuntos da campanha anterior. Só o primário deve mudar: confira energia total, direção, atmosfera, cortes e modelos nas configurações e nos `Header`. Execute na raiz do repositório baixado, com `A` e `B` apontando para os arquivos:

comparacao com PyROOT bash / cód. 7.10

```bash
python python/conex.py compare "$A" "$B" --output comparison-python
```

No GitHub: `python/conex.py`.

O script gera `comparison-python.pdf` e `comparison-python.csv`. Os grupos usam as mesmas bordas de bins. O rótulo `proton` corresponde ao primeiro arquivo, e `ferro` ao segundo: preserve essa ordem.

7.3 O que os números resumem

Para os valores aceitos x_i, calculamos diretamente:

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}, \qquad u(\bar{x}) \simeq \frac{s}{\sqrt{n}}.$$

O desvio amostral s descreve a dispersão entre eventos. $u(\bar{x})$ estima a incerteza estatística da média para eventos independentes com variância finita. Não são a mesma coisa, e nenhum dos dois resume os efeitos sistemáticos dos modelos ou da atmosfera.

Como exemplo resolvido, uma realização dessa pequena campanha produziu:

Grupo	n	Média	Desvio s	Interpretação
Próton	12	729,28	108,56	Dispersão maior nesta pequena amostra.
Ferro	12	580,93	29,12	Máximo médio em menor profundidade nesta amostra.

Média e desvio estão em $\mathrm{g\,cm^{-2}}$. As flutuações e a estatística pequena tornam os valores dependentes da realização. A comparação exercita o fluxo; não constitui uma conclusão de precisão sobre composição de raios cósmicos. Não ajustamos uma gaussiana para obter essas médias.

Se os conjuntos tiverem tamanhos diferentes, contagens brutas não comparam probabilidades. Use a mesma normalização e confira os bins externos antes de apresentar frações ou densidades. Aumentar o número de eventos exige mais tempo e armazenamento; comece com uma amostra pequena para estimar os recursos.

EXERCÍCIO 7.1 / Para praticar, sem gabarito

1. Compare o desvio amostral e o erro da média no CSV e explique o papel de cada um.
2. Altere os bins na função hist do script e verifique se o resumo dos valores individuais permanece igual. Use outro prefixo de saída.
3. Planeje uma comparação entre duas energias e indique quais parâmetros devem ficar fixos.

REFERÊNCIA / `python/conex.py`: compare; `results/comparison-summary.csv` e procedência dos resultados.

07 / ANÁLISE EM PYTHON [continuação] Guia / APOEMA

Ler os resultados com PyROOT

Qual máximo foi calculado?

OBJETIVO Distinguir estimadores e interpretar parâmetros de ajuste com suas limitações.

Branch	Interpretação nesta saída
`Xmax, Nmax`	Máximo e amplitude do ajuste Gaisser–Hillas estendido do perfil de partículas carregadas.
`Xmx, Nmx`	Máximo e amplitude obtidos por interpolação quadrática local de $N(X)$.
`XmxdEdX, dEdXmx`	Máximo e amplitude da interpolação quadrática local do depósito de energia.
`X0, p1, p2, p3`	Parâmetros da forma Gaisser–Hillas estendida. X0 não é Xfirst.

Se usar o ajuste já fornecido pelo CONEX, a função descrita no README é

$$N(X) = N_{\max}\left(\frac{X - X_0}{X_{\max} - X_0}\right)^{(X_{\max}-X_0)/\lambda(X)} \exp\left[\frac{X_{\max} - X}{\lambda(X)}\right], \qquad \lambda(X) = p_1 + p_2 X + p_3 X^2. \tag{7.1}$$

$N_{\max}$ é contagem; X, X_0, $X_{\max}$ e p_1 estão em $\mathrm{g\,cm^{-2}}$; p_2 é adimensional e p_3 tem unidade inversa de profundidade. A expressão é usada no domínio $X > X_0$, com $X_{\max} > X_0$ e $\lambda(X) > 0$ para a interpretação usual. Não avalie automaticamente fora desse domínio.

7.4 Critérios mínimos antes de comparar ajustes

Confirme `ixmax=1`; cheque valores finitos, máximo dentro do intervalo simulado e perfil com pontos antes e depois dele. Compare visualmente a curva com os pontos, especialmente se houver estrutura dupla ou corte no solo. `chi2` é descrito como $\chi^2/\mathrm{ndf}/\sqrt{N_{\max}}$; não tem a escala de um qui-quadrado reduzido convencional. Um valor pequeno não autoriza declarar ajuste excelente ou calcular um valor-p sem um modelo de erros.

Para os histogramas deste percurso usaremos `XmxdEdX`, sem novo ajuste de função ao histograma. É importante nomear o estimador no eixo e no texto: "máximo do depósito por interpolação quadrática", em vez de chamar todos os campos simplesmente de Xmax.

> **EXEMPLO 7.1 / Quando dois máximos não precisam coincidir**
>
> O máximo de $N(X)$ conta partículas acima de cortes; o máximo do depósito pondera a energia transferida ao meio. Trocar cortes pode mudar o primeiro sem reproduzir a mesma alteração no segundo. Uma diferença não demonstra, por si só, um bug. Primeiro verifique qual branch e qual perfil foram comparados.

REFERÊNCIA / README: steering files e descrição de Shower; CxRoot.cc:interpolateProfile; plotProfile.C:GaisserHillas.

07 / ANÁLISE EM PYTHON [continuação]
Guia / APOEMA

Ler os resultados com PyROOT

Interações líderes: o que salvar

OBJETIVO Produzir um arquivo com a árvore opcional e distinguir os dois contadores.

O primeiro arquivo do guia foi gerado sem `-K`: ele não contém `LeadingInteractions`. A árvore só é escrita quando o suporte está habilitado na compilação e a execução solicita `-K` maior que zero. Na distribuição estudada, o suporte já está habilitado; o padrão de execução é `-K 0`.

Vamos salvar até duas interações sucessivas da cadeia líder. Após cada colisão, o acompanhamento segue a partícula líder mais energética; não registra todas as ramificações nem gerações completas do chuveiro. Faça esta simulação uma vez. Se já produziu o arquivo, use-o e pule o bloco; se a pasta existir sem o arquivo esperado, escolha outro nome antes de continuar.

Execute no terminal Linux, com o ambiente e `CX` preparados:

um evento com detalhe da cadeia lider — bash / cód. 7.11

```bash
cd "$CX"
mkdir leading-run
export CONEX_ROOT="$CX/cfg"
export ROOT_OUT="$CX/leading-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 424242 -K 2 -x leading > leading-run/run.log 2>&1
```

O novo arquivo se chama `leading_sibyll23e_000424242_100.root`. O link leva à receita em `results`; gere o arquivo localmente. Confira o log e abra a saída antes de analisar. Uma entrada da árvore corresponde a um evento; dentro dela, `nInt` informa quantas interações foram efetivamente registradas, e `nPart` conta as partículas da lista armazenada. O número solicitado em `-K` não garante que a cadeia tenha sempre esse comprimento.

7.5 Branches por interação

Branch	Tipo/dimensão	Significado e unidade
`nInt`	`Int_t`	Número de interações registradas neste evento.
`mult`	`Int_t[nInt]`	Multiplicidade de secundárias por interação; contagem.
`pId`	`Int_t[nInt]`	Código da partícula que iniciou cada interação.
`pEnergy`	`Double_t[nInt]`	Energia dessa partícula, em GeV.
`matg`	`Int_t[nInt]`	Número de massa do núcleo-alvo.
`depth`	`Double_t[nInt]`	Profundidade da interação, em $\mathrm{g\,cm^{-2}}$.
`height`	`Double_t[nInt]`	Altitude da interação, em metros.
`kinel`	`Double_t[nInt]`	Campo de inelasticidade, adimensional; veja o cuidado abaixo.

`kinel` usa $1 - E_{\mathrm{maior}} / \sum E_{\mathrm{secundárias}}$ quando o cálculo está ativo. Nesta implementação, esse cálculo depende da inicialização da primeira interação; zeros nas interações seguintes não devem ser interpretados como colisões perfeitamente elásticas. Para estudar essa grandeza, comece pela primeira interação.

REFERÊNCIA / CxRoot.cc: LeadingInteractions; leadingInteractionsData.cc; conex_sub.F: cxafinal e controle de lxfirstIn.

07 / ANÁLISE EM PYTHON [continuação]
Guia / APOEMA

Ler os resultados com PyROOT

Secundárias e sua interação de origem

OBJETIVO Associar partículas a interações sem confundir os índices dos vetores.

Branch	Tipo/dimensão	Significado e unidade
`nPart`	`Int_t`	Total de partículas da lista armazenada no evento.
`Energy`	`Double_t[nPart]`	Energia de cada secundária, em GeV.
`Type`	`Int_t[nPart]`	Código de cada secundária.
`idInt`	`Int_t[nPart]`	Número da interação de origem: começa em 1 nesta cópia.
`px, py, pz`	`Double_t[nPart]`	Componentes do momento em GeV/c, quando habilitadas na compilação.

Para o vetor de multiplicidades, o índice 0 representa a primeira interação. Já `idInt[j] == 1` seleciona as partículas dessa primeira interação; `idInt[j] == 2`, as da segunda. Não use o índice da partícula para acessar diretamente um vetor de comprimento `nInt`.

As partículas são agrupadas por interação e ordenadas por energia na saída desta cópia. Use `idInt` para fazer a associação, em vez de depender da posição na lista. `mult[i]` descreve uma interação; `nPart` reúne a lista inteira. Sem truncamento, contar os valores de `idInt` de cada grupo permite conferir a multiplicidade. Se houver aviso de limite de armazenamento no log, essa igualdade precisa ser reavaliada.

> **ATENÇÃO / Códigos diferentes dos usados na entrada**
>
> Com `LEADING_INTERACTIONS_CORSIKA`, habilitado nesta cópia, `pId` e `Type` usam a convenção CORSIKA. Não os interprete automaticamente como códigos PDG nem como os códigos de núcleos aceitos em `-p`. Confira a convenção antes de identificar uma espécie pelo número.

Os nomes também exigem atenção: a branch `mult` contém a folha `Mult`, e `matg` contém `Matg`. A folha é o objeto que descreve o tipo e a dimensão do valor guardado em uma branch. Obtenha-a pela lista de folhas da branch quando a grafia for diferente.

Ler os resultados com PyROOT

Ler as multiplicidades em Python

Volte à raiz do guia e defina o arquivo novo no terminal:

arquivo com a cadeia lider — bash / cód. 7.12

```bash
export LIDER="$CX/leading-run/leading_sibyll23e_000424242_100.root"
python
```

No GitHub: receita dos dados (lideres).

No Python, abra a árvore e conte as secundárias de cada interação:

multiplicidade e associacao pela origem — Python / cód. 7.13

```python
import os, ROOT
from collections import Counter
f = ROOT.TFile.Open(os.environ["LIDER"], "READ")
t = f.Get("LeadingInteractions")
if not t:
    raise RuntimeError("Missing tree: check -K and the build")

t.Print()
t.GetEntry(0)
origens = Counter(int(t.idInt[j]) for j in range(int(t.nPart)))
for i in range(int(t.nInt)):
    print(i+1, int(t.mult[i]), origens[i+1])

f.Close()
exit()
```

Cada linha mostra a interação, a multiplicidade e a quantidade de partículas associadas. No exemplo de referência, aparecem 115 secundárias na primeira interação e 571 na segunda: `nInt = 2` e `nPart = 686`. Os índices dos vetores começam em 0, mas os valores de `idInt` começam em 1.

Para repetir a leitura com verificações e salvar o resumo por interação, execute:

resumo das interacoes com PyROOT — bash / cód. 7.14

```bash
python python/conex.py interactions "$LIDER" --output interactions-python
```

No GitHub: `python/conex.py`.

O CSV registra multiplicidade, contagem na lista, energia do projétil, inelasticidade, alvo, profundidade e altitude. O script obtém a folha pela branch, evitando confundir `mult` com `Mult`. Se escolher esta rota, pode continuar em Uso cotidiano; a próxima seção repete as operações usando C++.

REFERÊNCIA / `python/conex.py`: read_interactions e interactions; fonte do CONEX indicada nas tabelas.

08 / ANÁLISE EM C++ Guia / APOEMA

Ler os resultados com C++

Alternativa à rota Python

OBJETIVO Descobrir a estrutura real do arquivo sem supor nomes herdados de outro manual.

> **CONCEITO 8.1 / Árvore e branch**
>
> Uma TTree organiza entradas (aqui, eventos ou metadados). Branch é uma coluna ou grupo de valores da árvore. Uma entrada pode conter um escalar ou um vetor: um perfil inteiro pertence a um evento.

Esta é a segunda maneira de ler os arquivos. Se concluiu a rota Python da seção 7, você pode pular esta seção e seguir para Uso cotidiano. As duas rotas cobrem estrutura, perfis, histogramas, comparação e interações líderes; não é preciso executar ambas.

Abra o terminal Linux na raiz do repositório baixado, com ROOT disponível e `CX` definido. Vamos usar o mesmo arquivo do primeiro chuveiro:

abrir a saida no interpretador C++ bash / cód. 8.1

```bash
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
root -l "$ARQ"
```

No GitHub: receita dos dados (primeiro).

O comando abre o ROOT e disponibiliza o arquivo como `_file0`. Agora execute o bloco C++ dentro do ROOT:

exploracao interativa C++ / cód. 8.2

```cpp
_file0->ls();
auto h = _file0->Get<TTree>("Header");
auto s = _file0->Get<TTree>("Shower");
h->Print();
s->Print();
s->GetEntries();
h->Show(0);
s->Show(0);
s->GetLeaf("X")->GetTypeName();
s->GetLeaf("X")->GetLeafCount()->GetName();
.q
```

As chamadas pressupõem que as árvores e folhas existam, após conferir `ls`/`Print`. Em uma macro reutilizável, teste ponteiros antes de acessar. `inspect.C` implementa essa proteção para as árvores. Na saída estudada, `X` é `Float_t` e seu contador é `nX`; outras distribuições devem ser inspecionadas novamente.

8.1 Ciclos não são quatro árvores independentes

Nomes como `Header;1` e `Header;2` são ciclos de chaves de persistência ROOT para o mesmo nome de objeto. Autosave e gravação final podem produzir ciclos. Não há uma regra de "quatro árvores padrão, duas de backup". `Get("Shower")` recupera o ciclo mais recente, e `Get("Shower;1")` seleciona um ciclo explicitamente. Não some as entradas de ciclos como se fossem eventos novos.

REFERÊNCIA / CxRoot.cc:rootOut; documentação oficial TTree e TFile [10, 11]; `macros/inspect.C`.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Ler os tipos e as dimensões

OBJETIVO Selecionar o observável certo e evitar buffers de tamanho incorreto.

Tabela 8.1: Parte do esquema implementado na saída 2.51 em modo 1D.

Árvore / branch	Tipo ROOT	Significado
Header / Seed1, Particle	Int_t	Semente inicial e código do primário.
Header / lgEmin, lgEmax	Double_t	Limites de $\log_{10}(E/\mathrm{eV})$.
Header / Version	Float_t	Identificação gravada pelo programa; distinta de OutputVersion.
Header / OutputVersion	Float_t	Versão de formato, independente da anterior.
Header / HEModel, LEModel	Int_t	Códigos dos modelos efetivos.
Header / HiLowEgy	Float_t	Transição dos modelos hadrônicos, GeV.
Shower / lgE, zenith, azimuth	Float_t	Energia logarítmica e ângulos em graus.
Shower / altitude	Double_t	Parâmetro de impacto em metros.
Shower / nX	Int_t	Número de pontos de cada perfil.
Shower / X, H, D	Float_t[nX]	Profundidade, altitude e distância ao longo do eixo.
Shower / N, Mu, Electrons	Float_t[nX]	Populações acima dos cortes relevantes.
Shower / Gamma, Hadrons	Float_t[nX]	Fótons e hádrons dos perfis próprios.
Shower / dEdX, dMu	Float_t[nX]	Depósito por profundidade e produção de múons.
Shower / EGround	Float_t[3]	Energia em GeV: e+γ, hádrons, múons no limite final.

`Float_t` e `Double_t` são números de precisão simples e dupla; `Int_t` é inteiro. `/F`, `/D` e `/I` nas leaflists indicam esses tipos. Não ligue um endereço de `double` a um branch `/F`. `TTreeReaderArray<Float_t>` lê arrays de comprimento variável e evita assumir um buffer de 5000 ou 20000 pontos.

8.2 Outros campos que exigem cuidado

`Xfirst` é a profundidade da primeira interação em $\mathrm{g\,cm^{-2}}$, `Hfirst` é sua altitude em metros, e `XfirstIn` é inelasticidade, não profundidade apesar do nome. `Seed2` e `Seed3` guardam contadores de estado do gerador, não dois novos números escolhidos pelo usuário. `cpuTime` está em segundos de CPU por evento e não inclui necessariamente todo o custo de inicialização. `SVNRevision`, ainda listado no README, não é criado na árvore pelo C++ atual.

REFERÊNCIA / CxRoot.cc:923–944 e 1042–1077; README; registros/esquema-root.txt quando gerado no teste.

> **ATENÇÃO / Aprender com as macros originais**
>
> `plotProfile.C` orienta a associação de branches e a posição do depósito. Mas seu preenchimento de TH1 usa índice zero (underflow), e outras macros têm sintaxe antiga ou procuram `FirstInteraction`. Use-as como referência histórica, conferindo cada operação. Não as trate como um teste automático da compatibilidade atual.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Representar o perfil de um evento

OBJETIVO Criar gráficos com unidades corretas e respeitar planos e intervalos.

A macro `profile.C` abre `Shower`, verifica a entrada, lê `nX`, `X`, `N` e `dEdX` com tipos explícitos e compara os comprimentos. O evento zero é a primeira entrada. Um `TGraph` representa pares ordenados; `TCanvas` é a superfície onde o gráfico é desenhado e exportado. O bloco abaixo é um recorte ilustrativo de `profile.C`: **não o execute isoladamente**. A macro completa define `t` e seleciona a entrada; execute-a pelo comando ao final desta página.

trecho ilustrativo de profile.C - nao executar isoladamente — C++ / cód. 8.3

```
TTreeReader r(t);
TTreeReaderValue<Int_t> n(r,"nX");
TTreeReaderArray<Float_t> x(r,"X"), ncar(r,"N"), dep(r,"dEdX");
// After selecting the entry and checking the reader:
for (int i=0; i+1<*n; ++i) {
  double atual = x[i], proximo = x[i+1];
  double meio = (atual+proximo)/2.;
  double largura = proximo-atual;
  // N[i] is at plane X[i]; dep[i] refers to the next interval.
}
```

8.3 A posição do depósito é diferente

O README e a macro dos autores indicam que `dEdX[i]` se refere ao meio do intervalo entre `X[i]` e `X[i+1]`. Há nX planos e $nX - 1$ intervalos definidos entre eles. Nossa macro usa apenas esses intervalos e os meios calculados ponto a ponto, sem extrapolar o último ponto nem assumir espaçamento uniforme.

Ela calcula $E_{\mathrm{dep,intervalos}} \simeq \sum_{i=0}^{nX-2} (dE/dX)_i \, (X_{i+1} - X_i)$, em GeV. A soma não deve ser automaticamente igual à energia inicial: há energia no limite final, energia invisível e possíveis convenções de perdas e fronteiras. Também não é uma calibração calorimétrica de detector.

8.4 Executar a macro completa

Depois de sair do interpretador com `.q`, execute no terminal, na raiz do repositório baixado:

gerar perfil e CSV com C++ — bash / cód. 8.4

```
root -l -b -q "macros/profile.C(\"$ARQ\",0,\"profile-cpp\")"
```

O argumento zero seleciona a primeira entrada. O prefixo `profile-cpp` produz o PDF e o CSV desse nome; use um prefixo novo para cada análise. O modo `-b` evita janelas, e `-q` encerra depois da macro.

8.5 O que a macro entrega

`profile-cpp.pdf` reúne os gráficos de $N(X)$ e depósito; `profile-cpp.csv` contém os valores lidos para inspeção local, com a última linha sem um intervalo de depósito associado. A chamada recebe um prefixo e recusa substituir arquivos anteriores desse prefixo; use uma pasta nova para cada análise. Os CSV por evento não são incluídos no pacote redistribuível por causa das condições do componente UrQMD.

REFERÊNCIA / `macros/profile.C` desta entrega; macro original plotProfile.C; README: profiles.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Histogramas e resumos simples

OBJETIVO Transformar eventos em uma distribuição sem confundir largura física e incerteza da média.

> CONCEITO 8.2 / Histograma
>
> Agrupa valores em intervalos, chamados bins. A altura pode ser contagem, fração ou densidade; diga qual é usada. No ROOT os bins normais começam em 1; o bin 0 é underflow e o bin final extra é overflow.

Um histograma de `XmxdEdX` reúne um máximo do depósito por evento; ele não é o perfil longitudinal de um único chuveiro. Vamos usar a pequena campanha de 12 prótons e 12 ferros a 10^{17} eV, verticais, com mesmas configurações e sementes distintas. Execute no terminal na raiz do repositório baixado:

amostra pequena e abertura no ROOT — bash / cód. 8.5

```bash
bash scripts/campaign.sh "$CX" "$PWD/my-campaign"
export A="$PWD/my-campaign/proton_sibyll23e_000271828_100.root"
export B="$PWD/my-campaign/iron_sibyll23e_000271829_5600.root"
root -l "$A"
```

No GitHub: `scripts/campaign.sh`; receita dos dados (campanha).

O destino deve ser novo; se a campanha já existe, pule a primeira linha e ajuste `A` e `B`. No interpretador, recupere a árvore com `auto s = _file0->Get<TTree>("Shower");`, confira `s->Print();` e então execute:

um histograma inspecionavel — C++ / cód. 8.6

```cpp
auto hX = new TH1D("hX",";Xmax dE/dX [g/cm^{2}];Eventos",30,0,1200);
s->Draw("XmxdEdX>>hX", "", "goff");
hX->Draw("HIST");
hX->GetEntries();
hX->GetBinContent(0);       // Underflow
hX->GetBinContent(31);      // Overflow: 30 normal bins
```

O código acima é uma exploração direta. A macro `compare.C` acrescenta verificações e exclui valores não finitos ou não positivos, contando-os separadamente. Não faz um ajuste gaussiano para extrair uma média. Calcula os resumos diretamente dos valores de todos os eventos aceitos, inclusive os fora da faixa desenhada. Saia do interpretador com `.q` e gere o PDF e o resumo CSV no terminal:

histograma e resumo com C++ — bash / cód. 8.7

```bash
root -l -b -q "macros/histogram.C(\"$A\",\"histogram-cpp\")"
```

No GitHub: `macros/histogram.C`; `compare.C (dependÃªncia)`.

A macro usa 30 bins entre 0 e $1200\,\mathrm{g\,cm^{-2}}$ e exige pelo menos dois eventos válidos para o desvio amostral. Os resumos consideram todos os valores aceitos, inclusive underflow e overflow.

REFERÊNCIA / `macros/histogram.C` e `compare.C`; documentação TH1 e TTree [12, 10].

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Comparar conjuntos com C++

OBJETIVO Comparar duas distribuições com as mesmas unidades, seleção e bins.

Vamos usar os arquivos de prótons e ferro da campanha anterior. Confira energia total, direção, atmosfera, cortes e modelos nas configurações e nos `Header`; só o primário deve mudar nesta comparação. No terminal, na raiz do repositório baixado, execute:

comparar e salvar os resumos — bash / cód. 8.8

```bash
root -l -b -q "macros/compare.C(\"$A\",\"$B\",\"comparison-cpp\")"
```

No GitHub: `macros/compare.C`.

O primeiro arquivo recebe o rótulo `proton` e o segundo, `ferro`. A macro gera `comparison-cpp.pdf` e `comparison-cpp.csv`; ela recusa saídas já existentes, então use um prefixo novo para repetir a análise.

8.6 Média, dispersão e erro da média

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}, \qquad u(\bar{x}) \simeq \frac{s}{\sqrt{n}}. \tag{8.1}$$

O desvio amostral s descreve a dispersão evento a evento. $u(\bar{x})$ estima a incerteza estatística da média para eventos independentes com variância finita. Não é uma incerteza sistemática nem uma estimativa estável de caudas com uma dúzia de chuveiros.

8.7 Comparações que preservam a pergunta

As duas amostras devem usar mesma energia, ângulo, cortes, atmosfera e modelos, mudando somente o fator em investigação. Use bins e limites iguais. Se tamanhos diferirem, contagens brutas não comparam probabilidades: normalize pela integral e diga se a área é fração ou densidade por unidade de profundidade. Não normalize antes de conferir underflow/overflow.

Como exemplo resolvido, uma realização da campanha de 12 eventos por primário produziu média de $729{,}28\,\mathrm{g\,cm^{-2}}$ e desvio amostral de $108{,}56\,\mathrm{g\,cm^{-2}}$ para prótons; para ferro, $580{,}93$ e $29{,}12\,\mathrm{g\,cm^{-2}}$, respectivamente. Nesta pequena amostra, o máximo médio do ferro ocorre em menor profundidade e sua dispersão é menor.

Esses números são uma demonstração do fluxo e dependem das realizações simuladas. Uma dúzia de chuveiros não sustenta conclusões de precisão sobre composição ou pequenas diferenças entre modelos. A média vem dos valores dos eventos, sem ajuste gaussiano ao histograma.

EXERCÍCIO 8.1 / Para praticar, sem gabarito

1. Use Print para encontrar o tipo e o contador do array Mu no arquivo que você produziu.
2. Mude o número de bins e explique o que deve permanecer igual no resumo calculado diretamente dos eventos.
3. Explique por que fazer a média de todos os valores do array X mistura uma pergunta diferente da média de XmxdEdX.

REFERÊNCIA / `macros/compare.C`; `results/comparison-summary.csv` e procedência dos resultados.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Qual máximo foi calculado?

OBJETIVO Distinguir estimadores e interpretar parâmetros de ajuste com suas limitações.

Branch	Interpretação nesta saída
`Xmax, Nmax`	Máximo e amplitude do ajuste Gaisser–Hillas estendido do perfil de partículas carregadas.
`Xmx, Nmx`	Máximo e amplitude obtidos por interpolação quadrática local de $N(X)$.
`XmxdEdX, dEdXmx`	Máximo e amplitude da interpolação quadrática local do depósito de energia.
`X0, p1, p2, p3`	Parâmetros da forma Gaisser–Hillas estendida. X0 não é Xfirst.

Se usar o ajuste já fornecido pelo CONEX, a função descrita no README é

$$N(X) = N_{\max}\left(\frac{X - X_0}{X_{\max} - X_0}\right)^{(X_{\max}-X_0)/\lambda(X)} \exp\left[\frac{X_{\max} - X}{\lambda(X)}\right], \qquad \lambda(X) = p_1 + p_2 X + p_3 X^2. \tag{8.2}$$

$N_{\max}$ é contagem; X, X_0, $X_{\max}$ e p_1 estão em $\mathrm{g\,cm^{-2}}$; p_2 é adimensional e p_3 tem unidade inversa de profundidade. A expressão é usada no domínio $X > X_0$, com $X_{\max} > X_0$ e $\lambda(X) > 0$ para a interpretação usual. Não avalie automaticamente fora desse domínio.

8.8 Critérios mínimos antes de comparar ajustes

Confirme `ixmax=1`; cheque valores finitos, máximo dentro do intervalo simulado e perfil com pontos antes e depois dele. Compare visualmente a curva com os pontos, especialmente se houver estrutura dupla ou corte no solo. `chi2` é descrito como $\chi^2/\mathrm{ndf}/\sqrt{N_{\max}}$; não tem a escala de um qui-quadrado reduzido convencional. Um valor pequeno não autoriza declarar ajuste excelente ou calcular um valor-p sem um modelo de erros.

Para os histogramas deste percurso usaremos `XmxdEdX`, sem novo ajuste de função ao histograma. É importante nomear o estimador no eixo e no texto: "máximo do depósito por interpolação quadrática", em vez de chamar todos os campos simplesmente de Xmax.

> **EXEMPLO 8.1 / Quando dois máximos não precisam coincidir**
>
> O máximo de $N(X)$ conta partículas acima de cortes; o máximo do depósito pondera a energia transferida ao meio. Trocar cortes pode mudar o primeiro sem reproduzir a mesma alteração no segundo. Uma diferença não demonstra, por si só, um bug. Primeiro verifique qual branch e qual perfil foram comparados.

REFERÊNCIA / README: steering files e descrição de Shower; CxRoot.cc:interpolateProfile; plotProfile.C:GaisserHillas.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Interações líderes: o que salvar

OBJETIVO Produzir um arquivo com a árvore opcional e distinguir os dois contadores.

O primeiro arquivo do guia foi gerado sem `-K`: ele não contém `LeadingInteractions`. A árvore só é escrita quando o suporte está habilitado na compilação e a execução solicita `-K` maior que zero. Na distribuição estudada, o suporte já está habilitado; o padrão de execução é `-K 0`.

Vamos salvar até duas interações sucessivas da cadeia líder. Após cada colisão, o acompanhamento segue a partícula líder mais energética; não registra todas as ramificações nem gerações completas do chuveiro. Faça esta simulação uma vez. Se já produziu o arquivo, use-o e pule o bloco; se a pasta existir sem o arquivo esperado, escolha outro nome antes de continuar.

Execute no terminal Linux, com o ambiente e `CX` preparados:

um evento com detalhe da cadeia lider — bash / cód. 8.9

```bash
cd "$CX"
mkdir leading-run
export CONEX_ROOT="$CX/cfg"
export ROOT_OUT="$CX/leading-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 424242 -K 2 -x leading > leading-run/run.log 2>&1
```

O novo arquivo se chama `leading_sibyll23e_000424242_100.root`. O link leva à receita em `results`; gere o arquivo localmente. Confira o log e abra a saída antes de analisar. Uma entrada da árvore corresponde a um evento; dentro dela, `nInt` informa quantas interações foram efetivamente registradas, e `nPart` conta as partículas da lista armazenada. O número solicitado em `-K` não garante que a cadeia tenha sempre esse comprimento.

8.9 Branches por interação

Branch	Tipo/dimensão	Significado e unidade
`nInt`	`Int_t`	Número de interações registradas neste evento.
`mult`	`Int_t[nInt]`	Multiplicidade de secundárias por interação; contagem.
`pId`	`Int_t[nInt]`	Código da partícula que iniciou cada interação.
`pEnergy`	`Double_t[nInt]`	Energia dessa partícula, em GeV.
`matg`	`Int_t[nInt]`	Número de massa do núcleo-alvo.
`depth`	`Double_t[nInt]`	Profundidade da interação, em $\mathrm{g\,cm^{-2}}$.
`height`	`Double_t[nInt]`	Altitude da interação, em metros.
`kinel`	`Double_t[nInt]`	Campo de inelasticidade, adimensional; veja o cuidado abaixo.

`kinel` usa $1 - E_{\mathrm{maior}} / \sum E_{\mathrm{secundárias}}$ quando o cálculo está ativo. Nesta implementação, esse cálculo depende da inicialização da primeira interação; zeros nas interações seguintes não devem ser interpretados como colisões perfeitamente elásticas. Para estudar essa grandeza, comece pela primeira interação.

REFERÊNCIA / CxRoot.cc: LeadingInteractions; leadingInteractionsData.cc; conex_sub.F: cxafinal e controle de lxfirstIn.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Secundárias e sua interação de origem

OBJETIVO Associar partículas a interações sem confundir os índices dos vetores.

Branch	Tipo/dimensão	Significado e unidade
`nPart`	`Int_t`	Total de partículas da lista armazenada no evento.
`Energy`	`Double_t[nPart]`	Energia de cada secundária, em GeV.
`Type`	`Int_t[nPart]`	Código de cada secundária.
`idInt`	`Int_t[nPart]`	Número da interação de origem: começa em 1 nesta cópia.
`px, py, pz`	`Double_t[nPart]`	Componentes do momento em GeV/c, quando habilitadas na compilação.

Para o vetor de multiplicidades, o índice 0 representa a primeira interação. Já `idInt[j] == 1` seleciona as partículas dessa primeira interação; `idInt[j] == 2`, as da segunda. Não use o índice da partícula para acessar diretamente um vetor de comprimento `nInt`.

As partículas são agrupadas por interação e ordenadas por energia na saída desta cópia. Use `idInt` para fazer a associação, em vez de depender da posição na lista. `mult[i]` descreve uma interação; `nPart` reúne a lista inteira. Sem truncamento, contar os valores de `idInt` de cada grupo permite conferir a multiplicidade. Se houver aviso de limite de armazenamento no log, essa igualdade precisa ser reavaliada.

> **ATENÇÃO / Códigos diferentes dos usados na entrada**
>
> Com `LEADING_INTERACTIONS_CORSIKA`, habilitado nesta cópia, `pId` e `Type` usam a convenção CORSIKA. Não os interprete automaticamente como códigos PDG nem como os códigos de núcleos aceitos em `-p`. Confira a convenção antes de identificar uma espécie pelo número.

Os nomes também exigem atenção: a branch `mult` contém a folha `Mult`, e `matg` contém `Matg`. A folha é o objeto que descreve o tipo e a dimensão do valor guardado em uma branch. Obtenha-a pela lista de folhas da branch quando a grafia for diferente.

08 / ANÁLISE EM C++ [continuação] Guia / APOEMA

Ler os resultados com C++

Ler as multiplicidades com C++

Na raiz do repositório baixado, defina o caminho e abra o arquivo no terminal:

abrir o evento com detalhe — bash / cód. 8.10

```bash
export LIDER="$CX/leading-run/leading_sibyll23e_000424242_100.root"
root -l "$LIDER"
```

No GitHub: receita dos dados (lideres).

No interpretador ROOT, execute:

obter a folha pela branch correta — C++ / cód. 8.11

```cpp
auto t = _file0->Get<TTree>("LeadingInteractions");
if (!t) throw std::runtime_error("Missing tree: check -K");
t->Print();
t->GetEntry(0);
auto b = t->GetBranch("mult");
auto m = static_cast<TLeaf*>(b->GetListOfLeaves()->At(0));
std::cout << m->GetName() << " " << m->GetTypeName() << std::endl;
for (int i=0; i<m->GetLen(); ++i) {
  std::cout << "interaction " << i+1 << ": " << m->GetValue(i) << std::endl;
}
.q
```

O nome da folha será `Mult`; o vetor tem `nInt` elementos. No exemplo de referência, as multiplicidades são 115 e 571, com `nInt = 2` e `nPart = 686`. Para conferir a contagem das partículas por `idInt` e salvar o resumo, execute no terminal:

resumo das interacoes com C++ — bash / cód. 8.12

```bash
root -l -b -q "macros/interactions.C(\"$LIDER\",0,\"interactions-cpp\")"
```

No GitHub: `macros/interactions.C`.

A macro verifica tipos, os dois comprimentos e a associação `idInt = i+1`. O CSV contém energia do projétil, multiplicidade, partículas na lista, inelasticidade, alvo, profundidade e altitude por interação. Continue em Uso cotidiano; você não precisa executar a rota Python para completar este percurso.

REFERÊNCIA / `macros/interactions.C`; fonte do CONEX indicada nas tabelas.

09 / ROTINA Guia / APOEMA

Uso cotidiano

Guardar o contexto é parte da simulação

OBJETIVO Repetir uma análise sem depender da memória de quem rodou.

Use uma pasta nova por campanha, com um manifesto que descreva modelo, primário, energia, ângulos, número de eventos, seed e configuração. Nome de arquivo ajuda, mas não substitui o manifesto: energia e direção não entram no nome automático desta interface.

organizacao sugerida / cód. 9.1

```
campaign-energy17/
  environment.txt          # Sistema, ferramentas, variaveis relevantes
  campaign.csv           # Parametros por run
  used-config.param    # Copia exata da configuracao lida
  proton.log, iron.log  # Um log por run
  *.root                 # Resultados locais, nao redistribuidos aqui
  SHA256SUMS             # Integridade de entradas e saidas
analysis-energy17/
  comparacao.pdf, comparacao.csv
  comando.txt            # Chamada exata da analise
```

9.1 Sementes e independência

Escolha seeds positivas distintas e registre-as. Reiniciar cada evento com a mesma seed pode repetir o evento. O run com `-n 12` avança o gerador. Para jobs independentes, guarde a correspondência entre job, seed e comando; não dependa apenas do relógio para evitar colisões em execuções simultâneas.

9.2 Logs que ajudam a investigar

`> run.log 2>&1` guarda saída normal e de erro. Leia o início (configuração e inicialização) e o fim (término e erros). O script usa `set -euo pipefail`: falhas em comandos encerram o fluxo. Se usar `tee`, preserve o código de saída do executável com `pipefail`; o sucesso de `tee` sozinho não valida o simulador.

9.3 Reunir arquivos sem perder metadados

`TChain` lê várias árvores de mesmo esquema. Confira os Header antes de misturar runs: uma entrada de Header por arquivo não se alinha com uma entrada de Shower por evento. A macro original `merge.C` ainda usa `FirstInteraction`; não foi adotada para a árvore atual.

> **EXERCÍCIO 9.1 / Para praticar, sem gabarito**
>
> 1. Crie um manifesto para três energias e duas seeds por energia sem lançar os jobs.
> 2. Explique como reconhecer uma duplicação acidental de eventos por seeds reiniciadas.
> 3. Defina quais informações um colega precisaria para refazer sua figura após você mover a pasta de simulação.

REFERÊNCIA / scripts/campaign.sh; Makefile; macros/merge.C; CxRoot.cc:Run e rootOut.

Uso cotidiano

Diagnóstico de compilação

OBJETIVO Usar o primeiro erro relevante como evidência, evitando mudanças aleatórias.

9.4 Ferramenta não encontrada [documentado/código]

`root-config: command not found`, `gfortran: not found` ou `make: not found`: use `command -v`, confira ativação do ambiente e versões. Não tente resolver uma ausência de compilador mudando parâmetros físicos. Em ambiente Conda, confira `CC`, `CXX`, `FC` e passe os wrappers ao Makefile quando necessário.

9.5 Cabeçalho ou biblioteca ausente [documentado/código]

`TFile.h: No such file` sugere ROOT não configurado ou include errado; `bzlib.h` ausente ou `cannot find -lbz2` sugere desenvolvimento de bzip2 incompleto. Compare `root-config --cflags --libs` e a linha real de compilação no log. Misturar ROOT binário e compilador incompatível pode causar falhas de ligação ou execução. Preserve o log antes de reconstruir.

9.6 Fortran antigo e compilador recente [documentado/código]

O Makefile usa `-std=legacy`, que já implica tolerância de algumas incompatibilidades de argumentos no GNU Fortran [13]. Não acrescente flags que suprimem diagnósticos sem ler o erro. Se precisar mudar uma receita ou fonte, registre exatamente o patch, o motivo e a versão do compilador; refaça o teste pequeno antes de rodar mais eventos. Isso não é evidência de validação física ampla.

9.7 Caminhos e instalação [código]

O executável guarda o prefixo usado para localizar `lib/Linux`. Copiar só `bin/conex2r` para outra pasta pode deixar o carregador apontando à instalação anterior. Caminhos com espaços também podem quebrar o Makefile ou a abertura Fortran de param. Trabalhe em uma cópia curta, sem espaços, e recompile após mudanças de prefixo. Em macOS há divergência de extensão de biblioteca; não há teste dessa plataforma nesta edição.

REFERÊNCIA / Makefile; ConexDynamicInterface.cc; documentação GNU Fortran [13]; logs de testes nos registros.

Uso cotidiano

Diagnóstico da execução e análise

OBJETIVO Separar falha de instalação, dado incompleto e escolha inadequada de observável.

Sintoma	Conferência verificável
`can not open CONEX shared library`	Caminho mostrado pelo carregador, existência da biblioteca do modelo e `ldd` nela. Um alvo de outro modelo não basta.
`InitConex: Error opening parameter file`	`CONEX_ROOT`, nome selecionado por -m, permissão de leitura e espaços no caminho.
`Cannot open file` nas tabelas	Nome exato impresso, existência e permissão; compare com as entradas fconex/fegs/furqmd do param gerado.
Mensagem de versão de tabela incompatível	Distribuição e conjunto de tabelas devem corresponder; não renomeie uma tabela de outro modelo para passar a checagem.
Arquivo ROOT vazio ou incompleto	Log inteiro, código de saída, abertura em READ, Header, número de entradas de Shower.
Branch não encontrado	Use ls e Print; confira maiúsculas e OutputVersion, além da diferença entre Header e Shower.
Leitura falha ou valores absurdos	Tipo da leaf, dimensão variável e contador nX; verifique se há branch opcional.
Histograma vazio	Entradas, expressão, seleção, valores não finitos e limites. Cheque underflow/overflow.
Máximo na borda	Intervalo simulado, solo e perfil antes/depois do máximo; não interprete ajuste fora do suporte.

9.8 Uma sequência de investigação

Reproduza com um evento e comandos registrados. Confirme o diretório e as variáveis; confira modelo, param e tabelas; leia o log; abra a saída; por fim rode a macro. Modifique um fator por vez. Guarde tanto o diagnóstico inicial quanto a mudança que resolveu. Não conclua "ROOT está quebrado" quando o arquivo nem chegou a ser escrito pelo simulador.

9.9 Custo e recursos

Tempo total inclui inicialização, leitura das tabelas, simulação e gravação. `cpuTime` por evento não é sinônimo de tempo de parede. Mais eventos, maior detalhe com -K e alterações de geometria ou limiares e massa do primário podem aumentar tempo e disco. Meça em uma amostra pequena e calcule o tempo de execução antes de planejar uma campanha grande. Não são fornecidos tempos universais por chuveiro.

REFERÊNCIA / Mensagens lidas em ConexDynamicInterface.cc, conex2r.F e Interface.bz2.cc; comportamento de leitura das macros desta entrega.

10 / CONSULTA
Guia / APOEMA

Glossário clicável

Física e ferramentas em poucas linhas

OBJETIVO Esclarecer termos do percurso e retornar ao ponto em que foram apresentados.

Os termos com caixas de conceito permanecem explicados no contexto. Aqui reunimos o vocabulário complementar; os links de retorno funcionam dentro do próprio PDF.

10.1 Física e geometria

Ângulo zenital. Ângulo entre a direção de chegada e a vertical local. Na convenção usada aqui, $0°$ indica incidência vertical descendente, $90°$ é horizontal e valores maiores que $90°$ descrevem chuveiros ascendentes.

Voltar ao texto.

Chuveiros atmosféricos extensos. Cascatas de partículas produzidas por interações e decaimentos na atmosfera. Seu desenvolvimento tem extensão longitudinal e lateral; o percurso deste guia usa os perfis ao longo do eixo calculados pelo CONEX.

Voltar ao texto.

Chuveiro ascendente. Chuveiro que se propaga para cima na atmosfera. Na convenção zenital desta distribuição, corresponde a ângulos acima de $90°$; escolher essa direção não cria suporte a primários ou processos que o programa não implementa.

Voltar ao texto.

CORSIKA. Programa de simulação detalhada de cascatas de partículas, com acompanhamento de trajetórias e interações. CORSIKA e CONEX são programas distintos; opções de integração entre eles não tornam intercambiáveis suas instruções de instalação e execução.

Voltar ao texto.

Raio cósmico. Partícula de origem extraterrestre que chega à atmosfera; prótons e núcleos são exemplos centrais neste guia. O primário incidente e as partículas secundárias produzidas no ar têm papéis diferentes na interpretação do chuveiro.

Voltar ao texto.

REFERÊNCIA / README; método híbrido [1, 2]; convenções de geometria e modelos da distribuição estudada.

Glossário clicável

Processos e método híbrido

Interações hadrônicas. Colisões que envolvem hádrons ou núcleos e a interação forte, produzindo secundárias. O CONEX combina modelos de alta e baixa energia; a escolha desses modelos influencia multiplicidades, energias e o desenvolvimento da cascata.

Voltar ao texto.

Equações de cascata. Equações de transporte que descrevem a evolução das populações de partículas em energia e profundidade, incluindo produção e perdas. No método híbrido, recebem termos-fonte da simulação Monte Carlo e tratam numericamente a parte de menor energia.

Voltar ao texto.

Monte Carlo. Método que usa sorteios para representar processos probabilísticos. No CONEX híbrido, a parte de maior energia é simulada dessa forma; as equações de cascata completam a evolução abaixo dos limiares escolhidos.

Voltar ao texto.

Termos-fonte. Contribuições que alimentam uma equação com partículas produzidas por outros processos. Na passagem Monte Carlo/equações de cascata, transportam a informação das partículas que deixam o tratamento explícito para a evolução numérica das populações.

Voltar ao texto.

Gaisser-Hillas. Função usada para ajustar a forma do perfil longitudinal e estimar seu máximo. O parâmetro X_0 do ajuste não é automaticamente a profundidade da primeira interação; a qualidade do ajuste deve ser conferida no perfil.

Voltar ao texto.

REFERÊNCIA / README; método híbrido [1, 2]; convenções de geometria e modelos da distribuição estudada.

10 / CONSULTA [continuação] Guia / APOEMA

Glossário clicável

Ferramentas e convenções

Bash. Shell e linguagem de comandos usada nos exemplos Linux. Interpreta variáveis como `$CX`, aspas, redirecionamentos e scripts; executar o mesmo texto em outra shell pode exigir adaptações de sintaxe.

Voltar ao texto.

Caminhos. Ao usar `cfg/conex_sibyll.param`, a procura começa no diretório atual, que pode ser conferido com `pwd`. Depois de `cd`, o mesmo caminho relativo pode apontar para outro lugar; `..` representa o diretório superior.

Voltar ao texto.

Flag. Opção escrita na chamada de um programa, frequentemente acompanhada de um valor, como `-n 1`. A mesma letra pode ter outro significado em outro executável; flags de execução e opções de compilação pertencem a etapas diferentes.

Voltar ao texto.

Fortran. Linguagem de programação usada nas rotinas físicas do CONEX e em modelos incorporados. Seus fontes precisam de um compilador Fortran; arquivos `.F` desta distribuição também passam pelo pré-processamento que seleciona trechos do código.

Voltar ao texto.

Macros. No ROOT, arquivos de código C++ executados para tarefas como ler dados e gerar gráficos. Já macros de compilação são símbolos do pré-processador que selecionam trechos de código: o sentido do termo depende do contexto.

Voltar ao texto.

PowerShell. Shell e linguagem de automação usada nos blocos identificados como Windows. Seus comandos e variáveis têm convenções próprias; abrir o Ubuntu no WSL muda o ambiente em que os comandos Linux devem ser executados.

Voltar ao texto.

WSL 2. Arquitetura do Windows Subsystem for Linux que executa um kernel Linux em uma máquina virtual leve. Permite usar uma distribuição Linux no Windows; executar CONEX nela continua sendo o percurso Linux, sem comprovar suporte nativo ao Windows.

Voltar ao texto.

PDG. Particle Data Group, colaboração que compila propriedades de partículas e mantém uma convenção de identificadores numéricos usada em simulações. No CONEX, núcleos de entrada usam $100A$; a saída detalhada pode usar códigos CORSIKA, que são outra convenção.

Voltar ao texto.

Tempo de parede. Tempo real decorrido entre o início e o fim de uma tarefa, como em um cronômetro. Inclui processamento e esperas, como leitura e gravação; por isso não coincide necessariamente com o tempo de CPU registrado por evento.

Voltar ao texto.

REFERÊNCIA / Documentação oficial de Bash, GNU Fortran, ROOT, PowerShell e WSL; convenção PDG [18].

11 / FONTES
Guia / APOEMA

Referências e aprofundamento

Fontes primárias e documentação oficial

OBJETIVO Localizar os fundamentos e conferir versões antes de atualizar o guia.

Comece pelo artigo de Bergmann et al. para o método e pelo de Pierog et al. para o contexto histórico. Os modelos listados nos artigos antigos não devem ser usados como catálogo da cópia 7.801. O README recomenda citar os dois trabalhos, com prioridade para o primeiro, e registra o trabalho original de Bossard et al. A lista abaixo preserva esses créditos.

Referências

[1] T. Bergmann, R. Engel, D. Heck, N. N. Kalmykov, S. Ostapchenko, T. Pierog, T. Thouw e K. Werner. *One-dimensional Hybrid Approach to Extensive Air Shower Simulation*. Astroparticle Physics 26 (2007), 420–432. DOI: 10.1016/j.astropartphys.2006.08.005. arXiv:astro-ph/0606564 (2006).

[2] T. Pierog et al. *First Results of Fast One-dimensional Hybrid Simulation of EAS Using CONEX*. Nuclear Physics B – Proceedings Supplements 151 (2006), 159–162. DOI: 10.1016/j.nuclphysbps.2005.07.029. arXiv:astro-ph/0411260 (2004).

[3] G. Bossard et al. *Cosmic ray air shower characteristics in the framework of the parton-based Gribov–Regge model NEXUS*. Physical Review D 63 (2001), 054030. arXiv:hep-ph/0009119. Trabalho original reconhecido pelo README.

[4] R. Ulrich, R. Engel e M. Unger. *Hadronic Multiparticle Production at Ultra-High Energies and Extensive Air Showers*. Physical Review D 83 (2011), 054026. DOI:10.1103/PhysRevD.83.054026. arXiv:1010.4310.

[5] Autores do cxroot/CONEX. Repositório e tags oficiais. Comparação documental: conex2r7.60 e conex2r7.801. Consulta em 23/10/2026.

[6] Air Shower Physics. corsika-data, repositório institucional de tabelas; árvore d8ac240e consultada como referência da tag 7.801. Consulta em 23/10/2026.

[7] KIT, CORSIKA. Downloads e condições de obtenção. Não confundir a versão do CORSIKA com a do executável CONEX independente. Consulta em 23/10/2026.

Referências e aprofundamento

Documentação de ferramentas e materiais locais

OBJETIVO Manter os atalhos de consulta sem depender deles para ler o PDF offline.

Referências

[8] ROOT/CERN. Installing ROOT. Métodos e ativação; a versão usada nos testes é registrada à parte. Consulta em 23/10/2026.

[9] ROOT/CERN. Dependencies. Requisitos por plataforma e compilador. Consulta em 23/10/2026.

[10] ROOT/CERN. TTree Class Reference. Estrutura e leitura de árvores. Consulta em 23/10/2026.

[11] ROOT/CERN. TFile Class Reference. Arquivos, leitura e persistência. Consulta em 23/10/2026.

[12] ROOT/CERN. TH1 Class Reference. Histogramas e convenção dos bins. Consulta em 23/10/2026.

[13] GNU Project. GNU Fortran: Fortran Dialect Options; Argument Passing Conventions. Consulta em 23/10/2026.

[14] Conda. Managing Environments. Consulta em 23/10/2026. Testes usaram Micromamba, com pacotes do canal conda-forge.

[15] Microsoft Learn. How to install Linux on Windows with WSL. Consulta em 23/10/2026.

[16] ROOT/CERN. Python Interface. Acesso aos objetos ROOT a partir de Python. Consulta em 25/10/2026.

[17] ROOT/CERN. TGraph Class Reference. Pares ordenados e gráficos. Consulta em 25/10/2026.

[18] Particle Data Group. Monte Carlo Particle Numbering Scheme, revisão 2026. Consulta em 25/10/2026.

11.1 Fontes locais e créditos técnicos

Marcos Olegário: *CONEX.md*, manual anterior, identificação confirmada pelo grupo.

Distribuição local: README, Makefile, CMakeLists.txt, fontes, paramin, macros e documentação dos auxiliares foram confrontados com os procedimentos. Autores do CONEX e da interface ROOT são reconhecidos conforme o README; o guia não atribui ao APOEMA a autoria do simulador. Em publicações, cite também os modelos efetivamente usados conforme orientação de seus autores e da distribuição.

As figuras de resultados foram geradas nos testes; o esquema híbrido é desenho conceitual original deste guia. Não foram utilizadas fotografias ou figuras externas sujeitas a licença de reprodução. Os textos dos documentos fornecidos foram tratados como material de estudo, não como ordens para instalar programas ou alterar o sistema.

12 / CONSULTA
Guia / APOEMA

Comandos Linux

Navegar, conferir arquivos e consultar ajuda

OBJETIVO Encontrar rapidamente a função dos comandos usados ao longo do guia.

As tabelas reúnem comandos Linux/Bash do texto e complementos comuns de navegação. Os exemplos são para consulta: substitua nomes e caminhos antes de executar. Instruções de Windows e chamadas do interpretador ROOT aparecem separadas ao final.

Comando ou exemplo	Função e uso no guia
`pwd`	Mostra o diretório atual; ajuda a interpretar caminhos relativos.
`ls`	Lista o conteúdo do diretório atual.
`ls -lh first-run`	Lista detalhes, permissões e tamanhos em unidades legíveis.
`cd "$CX"`	Entra no diretório guardado em CX.
`cd ..`	Sobe um nível na estrutura de diretórios.
`cd`	Volta à pasta pessoal do usuário Linux.
`mkdir first-run`	Cria a pasta indicada; sem -p, avisa se ela já existir.
`cp arquivo copia`	Copia um arquivo; a origem permanece. cp -r copia diretórios.
`mv antigo novo`	Renomeia ou move um arquivo/diretório. Confira o destino para preservar dados.
`rm -i arquivo`	Remove o arquivo após pedir confirmação; complemento comum para organizar resultados.
`cat arquivo`	Mostra o conteúdo de um arquivo de texto; no guia, identifica a distribuição Linux.
`head -n 20 arquivo`	Mostra as primeiras 20 linhas, como no início de um arquivo param.
`tail -n 25 run.log`	Mostra as últimas 25 linhas do log.
`less run.log`	Abre texto para navegação; use q para sair. Complemento útil para logs longos.
`man ls` / `help cd`	Consultas separadas: abre o manual de ls ou mostra a ajuda interna de cd no Bash.

REFERÊNCIA / GNU Coreutils e Bash Reference Manual.

Comandos Linux

Conferir o sistema e preparar ferramentas

OBJETIVO Consultar os comandos de diagnóstico, instalação e ativação.

Comando ou exemplo	Função e uso no guia
`uname -a`	Mostra informações do kernel, sistema e arquitetura.
`command -v make`	Mostra como a shell encontra Make. Pode receber vários nomes de ferramentas.
`gcc -version`	Mostra a versão do compilador C; g++ e gfortran fazem a consulta equivalente para C++ e Fortran.
`"$CXX" -version`	Executa o compilador indicado por CXX; para Fortran, o guia usa `"$FC" -version`.
`root-config -version`	Mostra a versão do ROOT disponível na sessão.
`root-config -cflags -libs`	Mostra opções de compilação e ligação exigidas por essa instalação ROOT.
`ldd biblioteca.so`	Lista dependências de uma biblioteca Linux para investigar carregamento.
`sudo apt update`	Atualiza o índice de pacotes do Ubuntu/Debian com privilégios administrativos.
`sudo apt install pacotes`	Instala os pacotes indicados; no guia, prepara dependências do sistema.
`bash arquivo.sh`	Executa um script com Bash; usado para o instalador e os scripts do projeto.
`source arquivo.sh`	Lê instruções na shell atual; usado para preparar uma instalação ou o Conda.
`conda create -n conex-apoema`	Cria o ambiente de trabalho separado.
`conda activate conex-apoema`	Ativa o ambiente; conda deactivate sai dele.
`conda install -c conda-forge root`	Instala ROOT pelo canal conda-forge no ambiente ativo.
`conda config -env ...`	Ajusta uma configuração do ambiente; o guia define `channel_priority strict`.
`conda -version`	Confere a disponibilidade e a versão do gerenciador Conda.

REFERÊNCIA / Bash e Coreutils; documentação oficial de Conda, ROOT e Ubuntu.

12 / CONSULTA [continuação] Guia / APOEMA

Comandos Linux

Compilar, executar e registrar resultados

OBJETIVO Distinguir programas executados no terminal de recursos da shell.

Comando ou exemplo	Função e uso no guia
`make sibyll`	Compila o alvo SIBYLL descrito no Makefile; make sem alvo mostra a ajuda deste pacote.
`make all / make clean`	Chamadas distintas: all prepara todos os modelos; clean remove produtos de compilação. Não são necessários para repetir o primeiro teste.
`bin/conex2r -h`	Mostra a ajuda do executável CONEX compilado.
`bin/conex2r [opÃğÃţes]`	Executa o simulador com as flags escolhidas. Ver seção 6.
`root -l arquivo.root`	Abre ROOT com o arquivo indicado para exploração interativa.
`root -l -b -q macro.C`	Executa uma macro ROOT sem janelas e encerra ao final; chamadas com argumentos exigem as aspas mostradas nos exemplos.
`python -c "import ROOT"`	Executa uma instrução Python para conferir o acesso ao PyROOT.
`python python/conex.py -help`	Mostra as operações de análise Python incluídas no projeto.
`export ROOT_OUT="pasta"`	Define a variável e a transmite aos programas iniciados nessa sessão.
`printf`	Mostra texto formatado; no exemplo, imprime o código de saída guardado em status.
`tee arquivo.log`	Mostra texto recebido e também o grava; em pipelines, confira o código de saída do simulador.
`set -euo pipefail`	Configura o tratamento de falhas nos scripts Bash do projeto.

REFERÊNCIA / README e Makefile do CONEX; Bash Reference Manual; documentação oficial de ROOT e pdfLaTeX.

Comandos Linux

Ler a sintaxe e distinguir os contextos

OBJETIVO Reconhecer símbolos da shell e chamadas que pertencem a outros programas.

Comando ou exemplo	Função e uso no guia
`CX="caminho"`	Atribuição de variável na shell. Não coloque espaços ao redor de =.
`$HOME / $CX`	Expansões de variáveis: a primeira contém a pasta pessoal; a segunda guarda o caminho escolhido neste guia.
`"$CX"`	Aspas duplas preservam o valor como um argumento, inclusive se houver espaços.
`status=$?`	Guarda o código de saída do comando anterior. Deve vir imediatamente após ele.
`> arquivo.log`	Redireciona a saída normal para um arquivo, substituindo seu conteúdo anterior.
`2>&1`	Envia a saída de erro para o mesmo destino da saída normal já configurado.
`|`	Liga a saída de um comando à entrada de outro, como ao usar tee.
`\`	No final da linha, continua o mesmo comando na próxima linha; não acrescente espaços nem texto depois da barra.
`# comentário`	Inicia um comentário fora de aspas; não integra o comando executado.

12.1 Consulta rápida de uma execução

1. Ambiente ativo: `command -v root-config` e versões dos compiladores.
2. Raiz de execução: `cd "$CX"`; configuração: `CONEX_ROOT="$CX/cfg"`.
3. Diretório novo para saída; nunca dependa só do prefixo para proteger resultados.
4. Modelo compilado e tabelas correspondentes disponíveis.
5. Defina ambos os limites de energia e ambos os zenitais.
6. Seed positiva, log e manifesto guardados.
7. Confira saída, Header, entradas e esquema antes dos gráficos.

12.2 Windows e interpretador ROOT

`wsl -install -d Ubuntu`, `wsl -list -verbose` e `wsl -d Ubuntu` são comandos apresentados para PowerShell/Windows. Já `ls()`, `Print()`, `Show()`, `Draw()` e `.q` pertencem ao interpretador ROOT nos exemplos: não são comandos Linux para digitar diretamente no Bash.

REFERÊNCIA / Bash Reference Manual; Microsoft/WSL; documentação ROOT citada na seção 7.